

Análise de Algoritmos II

Mario Leston-Rey

June 27, 2026

Contents

1	Basic graph theory	1
1.1	Neighborhood of a vertex	2
1.2	Cut and degree of a vertex	2
1.3	Subgraphs and supergraphs	4
1.4	Walks, paths and circuits	6
1.4.1	Connected components	7
1.5	Acyclic graphs	8
1.6	Trees	10
1.7	Cuts of a graph	12
2	Basic greedy algorithms	14
2.1	Interval covering	16
2.1.1	The problem	16
2.1.2	A recursive approach	16
2.1.3	An iterative approach	18
2.1.4	A min-max approach	19
2.1.5	An implementation	21
2.2	The task scheduling problem	23
2.2.1	The problem	23
2.2.2	A recursive algorithm	23
2.2.3	An iterative approach	27
2.2.4	An implementation	29
2.2.5	A min-max approach	31
2.2.6	Longest path approach	36
2.3	Scheduling all tasks	39
2.4	Huffman coding	43
2.4.1	The tree representation	54
2.5	The fractional knapsack problem	58
2.6	More exercises	62

3	Minimum spanning trees	65
3.1	Spanning trees	65
3.2	The minimum spanning tree problem	68
3.3	Kruskal-Boruvka algorithm	71
3.4	Exercises	72

Chapter 1

Basic graph theory

To begin, let us define our main character. A **graph** G is an ordered pair (V, E) , where V is a finite and nonempty set whose elements are called **vertices**, and E is a set of two-element subsets of V ; the elements of E are called **edges**. So each edge is a pair $\{u, v\}$ where u and v , called the **endpoints** of the edge, are distinct vertices of V .¹ The vertex set of a graph G is often denoted by $V(G)$, and its edge set by $E(G)$. When the graph G is clear from the context, we often write V and E instead of $V(G)$ and $E(G)$, respectively. Moreover, we write uv instead of the longer $\{u, v\}$, usually when $\{u, v\}$ is an edge of G .

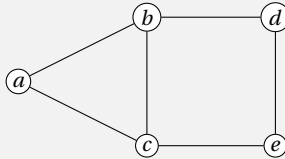


Figure 1.1: A graph with vertex set $\{a, b, c, d, e\}$ and edge set $\{ab, ac, bc, bd, ce, de\}$.

As the following definitions illustrate, many graph-theoretic concepts are highly intuitive and formalize everyday relationships among real-world entities. This helps explain their appeal and their wide range of practical applications.

¹With this convention, graphs have at least one vertex. This avoids some degenerate cases that would otherwise have to be excluded repeatedly.

1.1 Neighborhood of a vertex

For a graph G , vertices u and v of G are *neighbors* (or *adjacent*) if $uv \in E(G)$. We also say that v is a *neighbor* of u in G . The *neighborhood* of $u \in V(G)$ is the set

$$N_G(u) := \{v \in V(G) \mid uv \in E(G)\},$$

and we write $N(u)$ when G is clear from context. So N_G is a function that takes a vertex as input and returns a set of vertices as output, that is,² $N_G : V \rightarrow 2^V$.

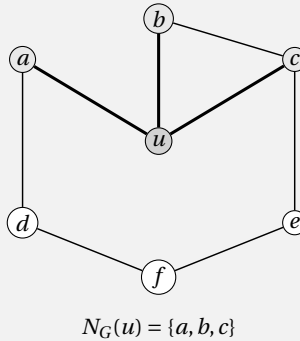


Figure 1.2: The neighborhood of the vertex u . The highlighted vertices are precisely the neighbors of u , that is, the vertices joined to u by an edge.

1.2 Cut and degree of a vertex

Let G be a graph, and let u be a vertex of G . The *cut* of u , denoted by $\delta_G(u)$, is the set of edges of G that are incident with u . Formally,

$$\delta_G(u) := \{e \in E \mid u \in e\}.$$

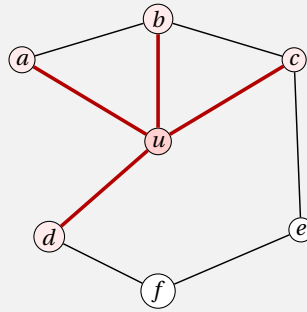
The notation $\delta_G(u)$ is somewhat heavy; when no confusion can arise, we simply write $\delta(u)$, or even just δu .

The *degree* of u , denoted by $\deg_G(u)$, or simply $\deg(u)$ when G is clear from the context, is the cardinality of δu . Thus

$$\deg_G(u) := |\delta_G(u)|.$$

Notice that $\deg_G(u) = |N_G(u)|$.

²For a set A , 2^A is the set of all subsets of A .



$$\begin{aligned}\delta_G(u) &= \{ua, ub, uc, ud\} \\ \deg_G(u) &= |\delta_G(u)| = 4\end{aligned}$$

Figure 1.3: The cut of a vertex. The highlighted edges are precisely the edges incident with u ; hence they form $\delta_G(u)$.

The next two results are usually among the first ones emphasized in introductions to graph theory, and we will keep to that tradition. We also take the opportunity to introduce a piece of notation that will be useful.

Let U be a set and let $X \subseteq U$. The *characteristic function of X relative to U* is the function

$$\mathbb{1}_U^X : U \rightarrow \{0, 1\}$$

defined by

$$\mathbb{1}_U^X(u) := \begin{cases} 1, & \text{if } u \in X, \\ 0, & \text{if } u \notin X, \end{cases}$$

for each $u \in U$. Notice that

$$|X| = \sum_{u \in U} \mathbb{1}_U^X(u).$$

For brevity, we will write $\mathbb{1}^X$ instead of $\mathbb{1}_U^X$ whenever the relevant universe is clear from the context. In the next lemma, we adopt this convention and write $\mathbb{1}^{\delta u}$ in place of $\mathbb{1}_E^{\delta u}$, where E is the edge set of the graph.

Lemma 1.1. *If G is a graph, then*

$$\sum_{v \in V(G)} \deg(v) = 2|E(G)|.$$

Proof. Let G be a graph. First observe that

$$\deg(v) = \sum_{e \in E} \mathbb{1}^{\delta v}(e)$$

for each $v \in V$. Moreover,

$$\sum_{v \in V} \mathbb{1}^{\delta v}(e) = 2$$

for each $e \in E$, since every edge has exactly two endpoints. Therefore,

$$\sum_{v \in V} \deg(v) = \sum_{v \in V} \sum_{e \in E} \mathbb{1}^{\delta v}(e) = \sum_{e \in E} \sum_{v \in V} \mathbb{1}^{\delta v}(e) = \sum_{e \in E} 2 = 2|E|.$$

□

Corollary 1.2. *Every graph has an even number of odd-degree vertices.*

Proof. Let G be a graph. Let P be the set of even-degree vertices of G , and let I be the set of odd-degree vertices of G . Clearly,

$$\sum_{v \in V} \deg(v) = \sum_{v \in P} \deg(v) + \sum_{v \in I} \deg(v).$$

By Lemma 1.1, the sum $\sum_{v \in V} \deg(v)$ is even. Also, $\sum_{v \in P} \deg(v)$ is even, since it is a sum of even integers. It follows that $\sum_{v \in I} \deg(v)$ is even. But this last sum is a sum of $|I|$ odd integers. Therefore $|I|$ is even. □

1.3 Subgraphs and supergraphs

A graph H is a *subgraph* of a graph G if

$$V(H) \subseteq V(G) \quad \text{and} \quad E(H) \subseteq E(G),$$

and, in this case, we write $H \subseteq G$ and we also say that G is a *supergraph* of H .

A subgraph H of G is called *spanning* if $V(H) = V(G)$. Thus a spanning subgraph of G is obtained by keeping all vertices of G and selecting some of its edges.

Let $\emptyset \neq X \subseteq V(G)$. The subgraph of G *vertex-induced* by X , or simply *induced* by X , denoted by $G[X]$, is the graph whose vertex set is X and whose edge set is

$$E(G[X]) := \{uv \in E(G) \mid u, v \in X\}.$$

In other words, $G[X]$ is obtained from G by keeping the vertices in X and all edges of G whose endpoints both belong to X .

If $X \subseteq V(G)$ and $V(G) \setminus X \neq \emptyset$, we write $G - X$ for the induced subgraph $G[V(G) \setminus X]$. We say that $G - X$ is obtained from G by the *deletion* of the vertices in X . In particular, if $u \in V(G)$ and $V(G) \setminus \{u\} \neq \emptyset$, we write $G - u$ instead of $G - \{u\}$. If $\emptyset \neq F \subseteq E(G)$, one may also consider the subgraph whose edge set is F and whose vertex set is the set of all endpoints of the edges in F , that is, the set $\bigcup F$.³ This graph is called the subgraph of G *edge-induced* by F , or simply *induced* by F , and is denoted by $G[F]$.

³If F is a set whose elements are also sets, then $\bigcup F$ is the set of all x for which there exists $f \in F$ with $x \in f$.

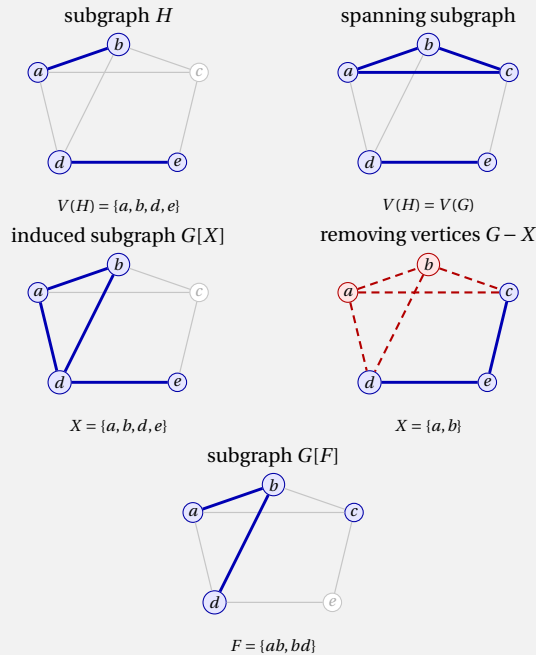


Figure 1.4: Different ways of obtaining subgraphs from a graph G . A subgraph may keep only some vertices and some edges whose endpoints are in the kept set of vertices. A spanning subgraph keeps all vertices. The induced subgraph $G[X]$ keeps all edges of G with both endpoints in X . The graph $G - X$ is obtained by removing the vertices in X and all edges incident with them. Finally, $G[F]$ is the subgraph generated by the edge set F ; its vertices are the endpoints of the edges in F .

Let G be a graph. If F is a set of two-element subsets of $V(G)$, we define

$$G + F := (V(G), E(G) \cup F).$$

Thus $G + F$ is the graph obtained from G by adding the edges in F , and so $G + F$ is a supergraph of G . In particular, if e is a two-element subset of $V(G)$, we write $G + e$ instead of $G + \{e\}$.

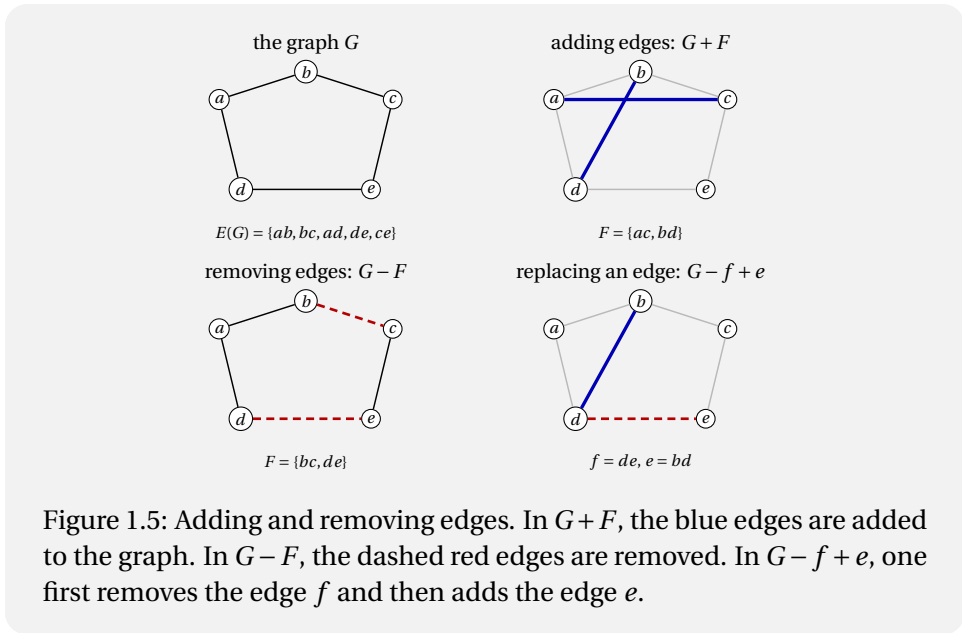
Similarly, if $F \subseteq E(G)$, we define

$$G - F := (V(G), E(G) \setminus F),$$

and thus $G - F$ is a subgraph of G . If $e \in E(G)$, we write $G - e$ instead of $G - \{e\}$. We will also combine these operations. For instance, if e is a two-element subset of $V(G)$ and $f \in E(G)$, then

$$G - f + e$$

means $(G - f) + e$, that is, the graph $G - f + e = (V(G), (E(G) \setminus \{f\}) \cup \{e\})$.



1.4 Walks, paths and circuits

Let G be a graph. For each $k \in \mathbb{N}$, a sequence⁴

$$W := \langle v_i \mid i \in [0 : k] \rangle$$

of vertices of G is a (v_0, v_k) -walk, or simply a *walk*, of G if

$$v_{i-1} v_i \in E(G)$$

for every $i \in [1 : k]$. The number k is called the *length* of W , which is also denoted by $\|W\|$. If $v_0 = v_k$, we say that W is *closed*. The set of edges of W , denoted $E(W)$, is the set $\{v_i v_{i+1} \mid i \in [0 : k]\}$. Similarly, the set of vertices of W , denoted $V(W)$, is the set $\{v_i \mid i \in [0 : k]\}$. We say that W *uses* an edge $e \in E(G)$ if $e \in E(W)$.

⁴Let $n \in \mathbb{N}$, and let Σ be a set. A *sequence* over Σ is a function $w : [0 : n) \rightarrow \Sigma$. It is *empty* if $n = 0$, and, in this case, we denote it by ϵ . The set of sequences over Σ is denoted by Σ^* , while the set of nonempty sequences is denoted by Σ^+ . Thus, $\Sigma^+ = \Sigma^* \setminus \{\epsilon\}$. If $u : [0 : m) \rightarrow \Sigma$ and $v : [0 : n) \rightarrow \Sigma$ are sequences over Σ , the *concatenation* of u and v , denoted by $u \cdot v$, is the sequence $uv : [0 : m + n) \rightarrow \Sigma$ defined by

$$(uv)_i := \begin{cases} u_i, & \text{if } i \in [0 : m), \\ v_{i-m}, & \text{if } i \in [m : m + n). \end{cases}$$

So a walk W in a graph G is an element of $V(G)^+ = \{[0 : k] \rightarrow V \mid k \in \mathbb{N}\}$.

A walk $\langle v_i \mid i \in [0 : k] \rangle$ is a **path** if its vertices are pairwise distinct, that is, if $v_i \neq v_j$ for all $i, j \in [0 : k]$ with $i \neq j$.

A **circuit** in G is a walk $\langle v_i \mid i \in [0 : k] \rangle$ of length $k \geq 3$ such that $v_0 = v_k$ and $\langle v_i \mid i \in [0 : k] \rangle$ is a path.

Proposition 1.3. *Let G be a graph, and let $x, y \in V(G)$. There is an (x, y) -walk in G if and only if there is an (x, y) -path in G .*

Proof. Since every path is a walk, it remains to prove that if there is an (x, y) -walk in G , then there is an (x, y) -path in G .

We prove this by induction on the length of the walk. Let $W := \langle v_0, \dots, v_k \rangle$ be an (x, y) -walk in G , so that $v_0 = x$ and $v_k = y$. If W is a path, there is nothing to prove. Suppose then that W is not a path. Hence there exist indices $i, j \in [0 : k]$, with $i < j$, such that $v_i = v_j$. Consider the sequence

$$W' := \langle v_0, \dots, v_i, v_{j+1}, \dots, v_k \rangle.$$

This sequence is still an (x, y) -walk in G . Moreover, W' has length strictly smaller than the length of W . By the induction hypothesis, there exists an (x, y) -path in G , as required. $\square$

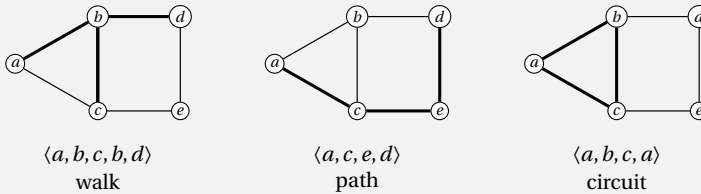


Figure 1.6: A walk, a path, and a circuit in a graph. In a walk, vertices may be repeated; in a path, all vertices are pairwise distinct; in a circuit, the first and last vertices coincide and the remaining vertices are pairwise distinct.

1.4.1 Connected components

Let G be a graph, and write $V := V(G)$. We define a binary relation on V , denoted by $\sim_G$ and called **reachability**, as follows: for $x, y \in V$, we set

$$x \sim_G y$$

if G contains an (x, y) -walk.

It is a simple matter to check that, for all $x, y, z \in V$, the following properties hold:

- (i) $x \sim_G x$;
- (ii) if $x \sim_G y$, then $y \sim_G x$;

(iii) if $x \sim_G y$ and $y \sim_G z$, then $x \sim_G z$.

Indeed, (i) follows from the walk $\langle x \rangle$. For (ii), it is enough to reverse an (x, y) -walk. For (iii), it is enough to concatenate an (x, y) -walk with a (y, z) -walk. Hence $\sim_G$ is an equivalence relation on V .

The equivalence classes of $\sim_G$ are called the **connected components** of G . Thus, for each $x \in V$, the connected component of G that contains x is the set

$$[x]_G := \{y \in V \mid x \sim_G y\}.$$

A graph G is **connected** if it has exactly one connected component. Equivalently, G is connected if, for every $x, y \in V(G)$, there exists an (x, y) -walk in G .

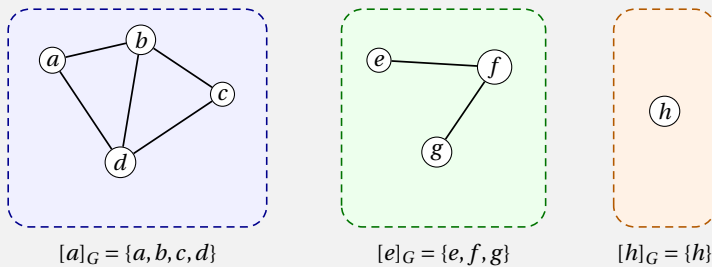


Figure 1.7: The connected components of a graph. Two vertices belong to the same component precisely when there is a walk between them. Thus the components are the equivalence classes of the reachability relation $\sim_G$.

1.5 Acyclic graphs

A graph is **acyclic** if it contains no circuit. A basic and intuitive property of acyclic graphs is stated below.

Lemma 1.4. *If a graph G is acyclic, then there is at most one path joining any two distinct vertices of G .*

Proof. We prove the contrapositive. Let G be a graph, and suppose that there are two distinct paths joining some pair of distinct vertices of G . We will show that G has a circuit.⁵ Among all such choices, choose $x, y \in V(G)$ and two distinct (x, y) -paths

$$P_1 := \langle u_0, u_1, \dots, u_k \rangle \quad \text{and} \quad P_2 := \langle v_0, v_1, \dots, v_\ell \rangle,$$

⁵This is very easy to see and intuitive. However, it requires some work to spell out the details of a rigorous proof.

with $u_0 = v_0 = x$ and $u_k = v_\ell = y$, in such a way that $k + \ell$ is as small as possible. We claim that $V(P_1) \cap V(P_2) = \{x, y\}$. Suppose not. Then there exist $i \in [1 : k]$ and $j \in [1 : \ell]$ such that $u_i = v_j$. Now, either the paths $\langle u_0, \dots, u_i \rangle$ and $\langle v_0, \dots, v_j \rangle$ are distinct, or the paths $\langle u_i, \dots, u_k \rangle$ and $\langle v_j, \dots, v_\ell \rangle$ are distinct. Indeed, if both pairs of paths were equal, then $P_1 = P_2$, contrary to our choice of P_1 and P_2 . In the first case, we obtain two distinct paths joining x and u_i , and their total length is $i + j < k + \ell$. In the second case, we obtain two distinct paths joining u_i and y , and their total length is $(k - i) + (\ell - j) < k + \ell$. In both cases, we contradict the choice of the vertices x, y and of the paths P_1, P_2 . Hence $V(P_1) \cap V(P_2) = \{x, y\}$. Consequently,

$$\langle u_0, u_1, \dots, u_k, v_{\ell-1}, v_{\ell-2}, \dots, v_0 \rangle$$

is a circuit of G . Therefore G is not acyclic. $\square$

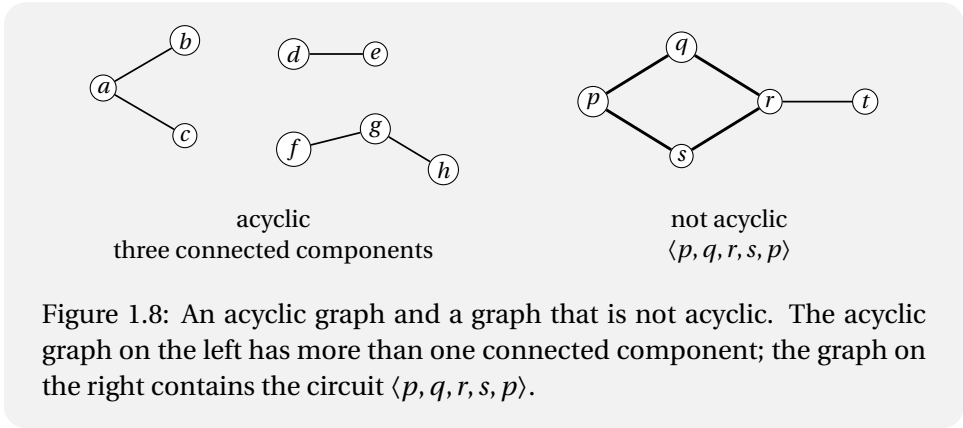
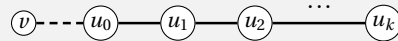


Figure 1.8: An acyclic graph and a graph that is not acyclic. The acyclic graph on the left has more than one connected component; the graph on the right contains the circuit $\langle p, q, r, s, p \rangle$.

Lemma 1.5. *If G is an acyclic graph and $|E(G)| \geq 1$, then there exists $u \in V(G)$ such that $\deg(u) = 1$.*

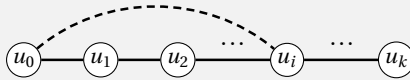
Proof. Suppose that G is an acyclic graph with $|E(G)| \geq 1$. Let $P := \langle u_0, \dots, u_k \rangle$ be a path of maximum length.⁶ Since $|E(G)| \geq 1$, there is an edge $uv \in E(G)$, and therefore $\langle u, v \rangle$ is a path of length 1. Hence $k \geq 1$. We claim that $\deg(u_0) = 1$. Since $k \geq 1$, the vertex u_0 has at least one neighbor, namely u_1 . Suppose that $\deg(u_0) \neq 1$. Then u_0 has a neighbor v distinct from u_1 . By the maximality of P , there exists $i \in [0 : k]$ such that $v = u_i$; otherwise, $\langle v, u_0, \dots, u_k \rangle$ would be a path longer than P . Since $v \neq u_1$ and v is a neighbor of u_0 , we have that $i \in [2 : k]$. But then $\langle u_0, \dots, u_i, u_0 \rangle$ is a circuit of G , contradicting the fact that G is acyclic. Therefore $\deg(u_0) = 1$. $\square$

⁶Such a path does exist. Indeed, consider the set $L := \{\|P\| \mid P \text{ is a path of } G\}$. The set L is nonempty, since $\langle u \rangle$ is a path for every $u \in V(G)$. Moreover, L is bounded above by $|V(G)| - 1$, because every path has length at most $|V(G)| - 1$. Thus L has a maximum, say k , and therefore there exists a path P of G with $\|P\| = k$.



$v \notin V(P)$ would give the longer path $\langle v, u_0, \dots, u_k \rangle$.

This is forbidden by the maximality of P .



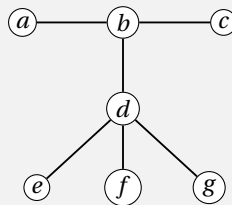
$v = u_i$, with $i \in [2 : k]$, would give the circuit $\langle u_0, \dots, u_i, u_0 \rangle$.

This is forbidden because G is acyclic.

Figure 1.9: Why an endpoint of a longest path in an acyclic graph has degree 1. Any neighbor of u_0 distinct from u_1 either extends the path or creates a circuit.

1.6 Trees

A graph is a **tree** if it is connected and acyclic. Thus, if T is a tree, then for every $x, y \in V(T)$ there exists a unique (x, y) -path in T . Indeed, existence follows from the fact that T is connected, and uniqueness follows from Lemma 1.4. This justifies the following notation. If T is a tree and $x, y \in V(T)$, we write $T(x, y)$ to denote the unique (x, y) -path in T .



connected and acyclic

Figure 1.10: A tree, that is, a connected and acyclic graph.

The next lemma will be useful for deriving some elementary properties of trees.

Lemma 1.6. *If G is a connected graph and u is a vertex of G of degree 1, then $G - u$ is also connected.*

Proof. Suppose that G is connected and that u is a vertex of degree 1. Let $G' := G - u$. We show that G' is connected.

Let $x, y \in V(G')$. Since G is connected, there exists an (x, y) -path P in G . Since $x, y \in V(G')$, the vertex u is not an end of P . Moreover, u cannot be an internal

vertex of P , because every internal vertex of P has degree at least 2 in G . Since u has degree 1 in G , it follows that u does not belong to P . Hence P is an (x, y) -path in G' , and therefore G' is connected. $\square$

Theorem 1.7. *If T is a tree, then $|E(T)| = |V(T)| - 1$.*

Proof. We prove the theorem by induction on $|V(T)|$. Let T be a tree. If $|V(T)| = 1$, then $E(T) = \emptyset$, and the result follows.

Suppose that $|V(T)| \geq 2$. By Lemma 1.5, T has a vertex u of degree 1. Consider the graph $T' := T - u$. It is clear that T' is acyclic, since a circuit in T' would be a circuit in T . Moreover, by Lemma 1.6, T' is connected. Thus T' is a tree. By the induction hypothesis, $|E(T')| = |V(T')| - 1$. Since u has degree 1, removing u removes exactly one edge. Hence $|E(T)| = |E(T')| + 1$ and $|V(T)| = |V(T')| + 1$. Therefore $|E(T)| = |E(T')| + 1 = (|V(T')| - 1) + 1 = |V(T')| = |V(T)| - 1$. $\square$

Theorem 1.8. *If T is an acyclic graph and $|E(T)| = |V(T)| - 1$, then T is a tree.*

Proof. We prove the theorem by induction on $|V(T)|$. Let T be an acyclic graph such that $|E(T)| = |V(T)| - 1$. If $|V(T)| = 1$, then T is connected and acyclic, and therefore T is a tree.

Assume that $|V(T)| \geq 2$. Then $|E(T)| = |V(T)| - 1 \geq 1$. By Lemma 1.5, T has a vertex u with $\deg(u) = 1$. Let $T' := T - u$. Since u has degree 1, we have $|E(T')| = |E(T)| - 1$. Moreover, $|V(T')| = |V(T)| - 1$. Hence $|E(T')| = |E(T)| - 1 = (|V(T)| - 1) - 1 = |V(T')| - 1$. Also, T' is acyclic, since a circuit in T' would be a circuit in T . By the induction hypothesis, T' is a tree. It remains to show that T is connected. Let $x, y \in V(T)$. If $x = y$, then there is nothing to prove. Suppose that $x \neq y$. If $x, y \in V(T')$, then, since T' is connected, there exists an (x, y) -path in T' , which is also an (x, y) -path in T . Suppose, finally, that one of x, y is equal to u , say $y = u$. Let v be the unique neighbor of u in T . Since $x \neq y$, we have $x \in V(T')$. Also, $v \in V(T')$. Since T' is connected, there exists an (x, v) -path P' in T' . Appending the vertex u to P' , that is, forming the path $P' \cdot \langle u \rangle$, we obtain an (x, u) -path in T . Therefore there exists an (x, y) -path in T . Thus T is connected. Since T is also acyclic, T is a tree. $\square$

Theorem 1.9. *If T is a connected graph and $|E(T)| = |V(T)| - 1$, then T is a tree.*

Proof. The proof is by induction on $|V(T)|$. Let T be a connected graph such that $|E(T)| = |V(T)| - 1$. If $|V(T)| = 1$, then $E(T) = \emptyset$, and therefore T is a tree.

Assume that $|V(T)| \geq 2$. Write $n := |V(T)|$ and $m := |E(T)|$. By Lemma 1.1, we have

$$\sum_{v \in V(T)} \deg(v) = 2m = 2n - 2.$$

Hence there exists $u \in V(T)$ with $\deg(u) \leq 1$. Indeed, if $\deg(v) \geq 2$ for every $v \in V(T)$, then

$$\sum_{v \in V(T)} \deg(v) \geq 2n,$$

which is impossible. Since T is connected and $n \geq 2$, no vertex of T has degree 0. Therefore $\deg(u) = 1$.

Let $T' := T - u$. By Lemma 1.6, the graph T' is connected. Moreover, since u has degree 1, removing u removes exactly one edge. Thus

$$|E(T')| = |E(T)| - 1 = (|V(T)| - 1) - 1 = |V(T)| - 2 = |V(T')| - 1.$$

By the induction hypothesis, T' is a tree.

The graph T is connected by hypothesis. It remains to show that T is acyclic. Since T' is a tree, T' has no circuit. If T had a circuit C , then u could not be a vertex of C , because every vertex of a circuit has degree at least 2 in T , while u has degree 1 in T . Hence C would be a circuit in T' , a contradiction. Hence T is acyclic and therefore T is a tree. $\square$

1.7 Cuts of a graph

Let G be a graph and let $X \subseteq V(G)$. We write $\delta_G(X)$, or simply $\delta(X)$ when the graph is clear from the context, to denote the set of edges of G with one endpoint in X and the other in $V(G) \setminus X$, that is,

$$\delta_G(X) := \{e \in E(G) \mid e \cap X \neq \emptyset \text{ and } e \cap (V(G) \setminus X) \neq \emptyset\}.$$

A **cut** of G is a subset K of $E(G)$ for which there exists a subset X of $V(G)$ such that $K = \delta_G(X)$.

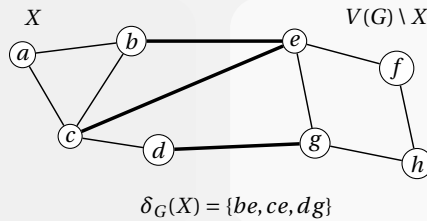


Figure 1.11: A cut of a graph. The highlighted edges are precisely the edges with one endpoint in X and the other in $V(G) \setminus X$.

Theorem 1.10. *A graph G is connected if and only if $\delta(X) \neq \emptyset$ for every nonempty proper subset X of $V(G)$.*

Proof. Assume first that G is connected. Let X be a nonempty proper subset of $V(G)$. Choose $x \in X$ and $y \in V(G) \setminus X$. Since G is connected, there exists an (x, y) -path $\langle u_0, \dots, u_k \rangle$ in G . Now $u_0 = x \in X$ and $u_k = y \in V(G) \setminus X$. Hence there exists $i \in [0 : k)$ such that $u_i \in X$ and $u_{i+1} \in V(G) \setminus X$. By the definition of $\delta(X)$, we have $u_i u_{i+1} \in \delta(X)$. Thus $\delta(X) \neq \emptyset$.

Conversely, assume that $\delta(X) \neq \emptyset$ for every nonempty proper subset X of $V(G)$. Let $x, y \in V(G)$, and let $X := [x]_G$. We claim that $\delta(X) = \emptyset$. Indeed, suppose that $u \in X$ and that v is a neighbor of u in G . Since $u \in X$, there exists an (x, u) -walk P in G . Now, $P \cdot \langle v \rangle$ is an (x, v) -walk in G . Thus $v \in X$. Therefore $\delta(X) = \emptyset$.

Since $x \in X$, the set X is nonempty. By the hypothesis, X cannot be a proper subset of $V(G)$. Hence $X = V(G)$. In particular, $y \in X$. Thus there exists an (x, y) -path in G . Therefore G is connected. $\square$

Chapter 2

Basic greedy algorithms

As an aside, throughout the text, a function from a set A to a set B is a subset $f \subseteq A \times B$ such that, for every $a \in A$, there exists a unique $b \in B$ with $(a, b) \in f$. This unique element is denoted by $f(a)$. In this case, we write $f : A \rightarrow B$. The domain of f , denoted by $\text{dom}(f)$, is A . Moreover, $\mathbb{R}_+$ denotes the set of nonnegative reals while $\mathbb{R}_{++}$ denotes the set of strictly positive reals.

We will explore a certain form of greedy algorithm. The basic idea is as follows. Suppose that we are given a finite set U and a collection of subsets of U , called *feasible* sets. To each feasible set we assign a cost, and we want to find a feasible set of maximum or minimum cost.

The algorithm is recursive and may be described informally as follows. If U is empty, simply return $\emptyset$; thus, we are implicitly assuming that the empty set is feasible. Otherwise, make a *greedy choice*—whatever that means in the problem under consideration—and select an element $u \in U$. Then reduce the problem to a proper subset $U' \subset U$ with the following property: whenever X' is a feasible subset contained in U' , the set $X' \cup \{u\}$ is feasible in the original problem. Recursively find a feasible subset X' of U' of maximum or minimum cost. Finally, return $X' \cup \{u\}$.

Algorithm 2.1 RECURSIVEGREEDY(U)

Require: A finite set U

Ensure: A feasible subset of U

- 1: **if** $U = \emptyset$ **then**
 - 2: **return** $\emptyset$
 - 3: choose $u \in U$ according to the greedy choice
 - 4: construct a proper subset $U' \subset U$
 - 5: $X' \leftarrow \text{RECURSIVEGREEDY}(U')$
 - 6: **return** $X' \cup \{u\}$
-

Of course, this description is only a scheme. It does not by itself guarantee that the algorithm returns an optimal solution. The difficulty is to prove that both the greedy choice and the reduction step are safe. In other words, we must prove that some optimal solution of the original problem can be obtained by extending an optimal solution of the reduced problem by the chosen element.

This recursive algorithm may also be implemented in an iterative way. We keep a set X of elements already chosen by the algorithm; initially, $X = \emptyset$. At each iteration, if $U = \emptyset$, we return X and stop. Otherwise, we make a greedy choice and select an element $u \in U$. We then update X to $X \cup \{u\}$, reduce the current problem to a proper subset $U' \subset U$, and begin the next iteration with U' in place of U and $X \cup \{u\}$ in place of X .

Algorithm 2.2 ITERATIVEGREEDYREDUCTION(U)

Require: A finite set U

Ensure: A feasible subset of the original set U

```

1:  $X \leftarrow \emptyset$ 
2: while  $U \neq \emptyset$  do
3:   choose  $u \in U$  according to the greedy choice
4:    $X \leftarrow X \cup \{u\}$ 
5:   construct a proper subset  $U' \subset U$ 
6:    $U \leftarrow U'$ 
7: return  $X$ 
  
```

There is another common way of organizing this greedy algorithm. First, we choose an ordering of the elements of U that reflects the greedy choices. The elements are then considered one by one. When an element is considered, the algorithm either adds it to the current solution or discards it permanently. The elements discarded in this process are precisely those that belong to the current instance but not to the reduced instance.

This process can be summarized as follows. The algorithm maintains a feasible set X , initially empty. At each step, it considers the next element u in the prescribed order. If adding u to the current set X preserves feasibility, or more generally if u passes the greedy test associated with the problem, then the algorithm updates X to $X \cup \{u\}$ and begins a new iteration. Otherwise, u is discarded and will never be considered again. When all elements have been inspected, the algorithm returns X .

Algorithm 2.3 GREEDYSCAN(U)

Require: A finite set U

Ensure: A feasible subset of U

```

1: choose an ordering  $u_0, \dots, u_{n-1}$  of the elements of  $U$ 
2:  $X \leftarrow \emptyset$ 
3: for  $r \leftarrow 0$  to  $n - 1$  do
4:   if  $X \cup \{u_r\}$  is feasible then
5:      $X \leftarrow X \cup \{u_r\}$ 
6: return  $X$ 

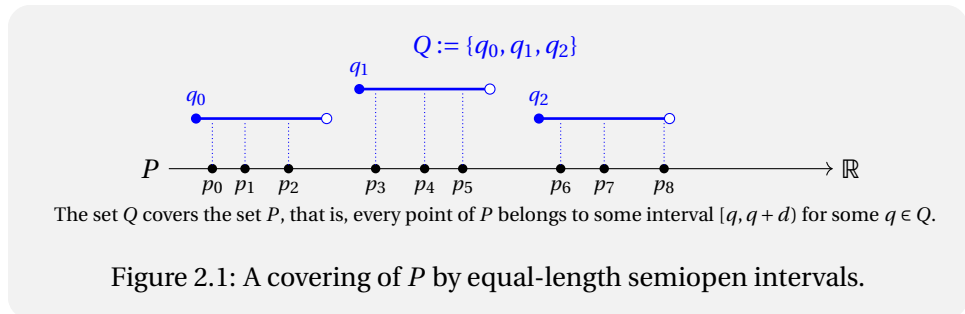
```

2.1 Interval covering

2.1.1 The problem

Let $d \in \mathbb{R}$ with $d > 0$. We say that a point $q \in \mathbb{R}$ *covers* a point $p \in \mathbb{R}$ if $q \leq p < q + d$, or equivalently, if¹ $p \in [q, q + d)$. We say that a subset Q of $\mathbb{R}$ *covers* a point $p \in \mathbb{R}$ if there exists $q \in Q$ such that q covers p . Moreover, we say that Q is a *covering* of $P \subseteq \mathbb{R}$ if Q covers every point in P .

We are interested in finding a covering of a finite subset P of $\mathbb{R}$ with the minimum possible number of points. Such a covering is called a *minimum covering* of P , or, more precisely, a *minimum covering* of (P, d) . This is the *equal-length interval covering problem*. Notice that P itself is a covering of P . Hence every finite subset of $\mathbb{R}$ has a finite covering.



2.1.2 A recursive approach

We will explore the idea of a greedy algorithm for this problem. Given a finite set of points P with $P \neq \emptyset$ —since for $P = \emptyset$ the solution is obvious—, we first make a *greedy choice*, namely, we choose the minimum element p of P . We then form

¹For $r, s \in \mathbb{R}$, $[r, s)$ denotes $\{x \in \mathbb{R} \mid r \leq x < s\}$.

the subproblem consisting of all points of P that are not covered by p . After recursively computing a minimum covering for this subproblem, we try to extend it by adding p . Thus, in order to justify the algorithm, we must prove that the covering obtained in this way is a minimum covering of the original set of points P . The next theorem shows that this strategy always works.

Theorem 2.1. *Let P be a nonempty finite subset of $\mathbb{R}$, and let $d \in \mathbb{R}$ with $d > 0$. Let p be the minimum element of P , and let $P' := \{q \in P \mid q \geq p + d\}$. If Q' is a minimum covering of P' , then $Q' \cup \{p\}$ is a minimum covering of P .*

Proof. Suppose that Q' is a minimum covering of P' . Let R be a finite covering of P , and let $Q := \{p\} \cup Q'$. We will show that $|Q| \leq |R|$.

First note that Q is a covering of P . Indeed, let $q \in P$. If $q < p + d$, then, since $p \leq q$, we have $p \leq q < p + d$, and so $\{p\}$ covers q . If $q \geq p + d$, then $q \in P'$, and so Q' covers q .

Since R is a covering of P , there exists $r \in R$ such that $r \leq p < r + d$. Let $\tilde{P} := \{q \in P \mid q \geq r + d\}$. Since p is the minimum element of P and $r \leq p$, we have $r + d \leq p + d$. Hence $P' \subseteq \tilde{P}$.

Now, $\tilde{R} := R \setminus \{r\}$ is a covering of $\tilde{P}$. Indeed, assume $q \in \tilde{P}$. Then $q \geq r + d$, so r does not cover q . Since R covers P , there exists $s \in R$ such that $s \leq q < s + d$. Since r does not cover q , we have $s \neq r$, and therefore $s \in \tilde{R}$. This proves that $\tilde{R}$ is a covering of $\tilde{P}$.

Moreover, since $P' \subseteq \tilde{P}$, the set $\tilde{R}$ is, a fortiori, a covering of P' . Since Q' is a minimum covering of P' , we have $|Q'| \leq |\tilde{R}|$. Also, $r \notin \tilde{R}$. Therefore

$$|Q| = |\{p\} \cup Q'| \leq |Q'| + 1 \leq |\tilde{R}| + 1 = |R|.$$

Since R was an arbitrary finite covering of P , we get that Q is a minimum covering of P , as required. $\square$

The theorem above immediately suggests the following recursive algorithm.

Algorithm 2.4 INTERVALCOVER(P, d)

Require: A finite set $P \subseteq \mathbb{R}$ and a real number $d > 0$

Ensure: A minimum covering of (P, d)

- 1: **if** $P = \emptyset$ **then**
 - 2: **return** $\emptyset$
 - 3: let $p \leftarrow \min P$
 - 4: $P' \leftarrow \{q \in P \mid q \geq p + d\}$
 - 5: $Q' \leftarrow \text{INTERVALCOVER}(P', d)$
 - 6: **return** $Q' \cup \{p\}$
-

Proof of correctness. The proof is by induction on $|P|$. The result is clear if $P = \emptyset$. Assume, then, that $P \neq \emptyset$. Since $d > 0$, we have $p \notin P'$, and hence $P' \subset P$. By the

induction hypothesis, Q' is a minimum covering of P' . By the theorem, $Q' \cup \{p\}$ is a minimum covering of P . $\square$

A more efficient implementation is obtained by sorting the points once and then scanning them from left to right. This gives the iterative version below.

2.1.3 An iterative approach

It is easy to obtain from the recursive algorithm a more efficient iterative implementation. First we order the points as $p_0, \dots, p_{n-1}$ so that $p_0 \leq \dots \leq p_{n-1}$. Then we scan this ordering from left to right, keeping only the last point chosen for the covering. If the current point is already covered by the last chosen point, we discard it; otherwise, we add it to the covering.

Algorithm 2.5 INTERVALCOVERFAST(P, d)

Require: A nonempty finite set $P \subseteq \mathbb{R}$ and a real number $d > 0$

Ensure: A minimum covering of (P, d)

```

1: let  $p_0, \dots, p_{n-1}$  be an ordering of  $P$  such that  $p_0 \leq \dots \leq p_{n-1}$ 
2:  $Q \leftarrow \{p_0\}$ 
3:  $\ell \leftarrow p_0$ 
4: for  $r \leftarrow 1$  to  $n - 1$  do
5:   if  $p_r \geq \ell + d$  then
6:      $Q \leftarrow Q \cup \{p_r\}$ 
7:      $\ell \leftarrow p_r$ 
8: return  $Q$ 

```

Complexity analysis Let $n := |P|$. The first step consists in ordering the points of P as $p_0, \dots, p_{n-1}$ so that $p_0 \leq \dots \leq p_{n-1}$. This can be done in $O(n \lg n)$ time.

The operations involving the set Q —initialization and insertion—can be implemented in $O(1)$ time. The loop has $n - 1$ iterations, and each iteration takes $O(1)$ time. Hence the loop takes $O(n)$ time. Therefore, the total running time of Algorithm 2.5 is $O(n \lg n)$.

Algorithm 2.5 was obtained from an operational and intuitive reasoning. We would like to establish its correctness directly. A natural candidate for such a direct proof would be the following claim.

Claim. Let P be a finite subset of $\mathbb{R}$, and let $d \in \mathbb{R}$ with $d > 0$. Let $P' \subset P$ be such that every point of P' is smaller than every point of $P \setminus P'$, and let $p := \min(P \setminus P')$. Suppose that $Q' \subseteq P'$ is a minimum covering of P' . Then:

- (i) if Q' covers p , then Q' is a minimum covering of $P' \cup \{p\}$;
- (ii) if Q' does not cover p , then $Q' \cup \{p\}$ is a minimum covering of $P' \cup \{p\}$.

However, this claim is false, as the following example shows. Let $d := 1$, let $P' := \{0, 0.9, 1.8\}$, and let $p := 1.9$. Then $Q' := \{0, 0.9\}$ is a minimum covering of P' . However, Q' does not cover $p = 1.9$. If the claim were true, then we would conclude that

$$Q' \cup \{p\} = \{0, 0.9, 1.9\}$$

is a minimum covering of $P' \cup \{p\}$. This is false, since $\{0, 1.8\}$ is a covering of $P' \cup \{p\}$.

We will give a rigorous proof of the correctness of Algorithm 2.5 in the next section, where we discuss a min-max approach to the problem.

2.1.4 A min-max approach

Consider a finite set P of points of $\mathbb{R}$. Suppose someone gives us a finite subset Q of $\mathbb{R}$ and asks us to prove that Q is a minimum covering of P . The first step is clear: we must check that Q is indeed a covering of P . But how could we certify that Q is minimum?

One possibility would be to run the algorithm, obtain a minimum covering, and compare its size with $|Q|$. This, however, is not very satisfactory: it proves minimality by appealing to the algorithm itself. We would like a more intrinsic certificate of minimality, one that can be checked without rerunning the algorithm.

We will now present a min-max approach to the problem. The idea is to produce not only a covering of P , but also another object that certifies the minimality of that covering. More precisely, this second object, called a *dual certificate*, will belong to a certain collection naturally associated with P , and its size will give a lower bound on the size of every covering of P .

Thus, if we construct a covering Q of P and such a certificate S with $|Q| = |S|$, then Q must be a minimum covering. Moreover, S must be maximum in its own collection. In this way, the problem is explained by a min-max relation: the minimum size of a covering is equal to the maximum size of a dual object, and the algorithm will produce both objects as output.

Let P be a finite subset of $\mathbb{R}$, and let $d \in \mathbb{R}$ with $d > 0$. We say that a subset $S \subseteq P$ is *d -separated* if the distance between any two distinct points of S is at least d , that is, if $|x - y| \geq d$ for every $x, y \in S$ with $x \neq y$.

necessary, that $s_1 \leq s_2$. Since q covers both s_1 and s_2 , Suppose that Q is a covering of P and that $S \subseteq P$ is d -separated. Let $q \in Q$, and suppose that q covers two distinct points $x, y \in S$. Assume, without loss of generality, that $y < x$. Then $q \leq y < x < q + d$, and hence $x - y < d$. This contradicts the fact that S is d -separated. Thus each element of Q covers at most one element of S . Since Q covers every element of S , it follows that $|Q| \geq |S|$.

We may thus conclude that

$$\min\{|Q| \mid Q \text{ is a covering of } P\} \geq \max\{|S| \mid S \subseteq P \text{ is } d\text{-separated}\}.$$

Lemma 2.2. *Let P be a finite subset of $\mathbb{R}$, and let $d \in \mathbb{R}$ with $d > 0$. If Q^* is a covering of P , $S^* \subseteq P$ is d -separated, and $|S^*| = |Q^*|$, then Q^* is a minimum covering of P and S^* is a maximum d -separated subset of P .*

Proof. Suppose that Q^* is a covering of P , that $S^* \subseteq P$ is d -separated, and that $|S^*| = |Q^*|$. Let Q be a covering of P , and let S be a d -separated subset of P . By the inequality proved above, we have

$$|Q| \geq |S^*| = |Q^*| \geq |S|.$$

Thus $|Q| \geq |Q^*|$ for every covering Q of P , and $|S^*| \geq |S|$ for every d -separated subset S of P . Therefore Q^* is a minimum covering of P , and S^* is a maximum d -separated subset of P . $\square$

Theorem 2.3. *Let P be a finite subset of $\mathbb{R}$, and let $d \in \mathbb{R}$ with $d > 0$. Then there exists a covering Q of P and a d -separated set $S \subseteq P$ such that $|Q| = |S|$.*

Proof. The proof is by induction on $|P|$. The result is trivial when $P = \emptyset$. Assume that $P \neq \emptyset$, and let $p := \min P$. Consider the set $P' := \{q \in P \mid q \geq p + d\}$. Since $d > 0$, we have $|P'| < |P|$. By the induction hypothesis, there exists a covering Q' of P' and a d -separated set $S' \subseteq P'$ such that $|Q'| = |S'|$.

Now, $Q := \{p\} \cup Q'$ is a covering of P , and $S := \{p\} \cup S'$ is a d -separated subset of P . Therefore

$$|Q| = 1 + |Q'| = 1 + |S'| = |S|.$$

This proves the theorem. $\square$

It is worth observing that the inductive construction can be carried out so that $Q = S$ at every step. Thus the proof actually produces a single set that is both a covering of P and a d -separated subset of P . By Lemma 2.2, this set is a minimum covering of P and a maximum d -separated subset of P .

Now we can come back to Algorithm 2.5 and give a rigorous proof of its correctness, which turns out to also give an alternative proof of Theorem 2.3. It depends on the next lemma.

Lemma 2.4. *Let P be a finite subset of $\mathbb{R}$, and let $d \in \mathbb{R}$ with $d > 0$. Let $P' \subset P$ be such that every point of P' is smaller than every point of $P \setminus P'$, and let $p := \min(P \setminus P')$. Suppose that $Q' \subseteq P'$ is a covering of P' that is d -separated. Then:*

- (i) *if Q' covers p , then Q' is a covering of $P' \cup \{p\}$ that is d -separated;*
- (ii) *if Q' does not cover p , then $Q' \cup \{p\}$ is a covering of $P' \cup \{p\}$ that is d -separated.*

Proof. For (i), suppose that Q' covers p . Since Q' already covers P' , it covers $P' \cup \{p\}$. Moreover, Q' is d -separated by hypothesis.

For (ii), suppose that Q' does not cover p . We first show that $Q' \cup \{p\}$ is d -separated. Let $q \in Q'$. Since $Q' \subseteq P'$ and every point of P' is smaller than every point of $P \setminus P'$, we have $q < p$. Since q does not cover p , it follows that $p \geq q + d$. Hence $p - q \geq d$. Thus p is at distance at least d from every point of Q' . Since Q' is d -separated, the set $Q' \cup \{p\}$ is d -separated.

Finally, $Q' \cup \{p\}$ covers $P' \cup \{p\}$, since Q' covers P' and p covers itself. Therefore $Q' \cup \{p\}$ is a covering of $P' \cup \{p\}$ that is d -separated. $\square$

Proof of correctness of the iterative version. For each $r \in [1 : n]$, let

$$P_r := \{p_0, p_1, \dots, p_{r-1}\}.$$

We claim that, at the beginning of each iteration $r \in [1 : n]$ of the loop in Algorithm 2.5, the following properties hold:

- (i) $\ell = \max Q$;
- (ii) $Q \subseteq P_r$ covers P_r and is d -separated.

At the beginning of the first iteration, we have $r = 1$, $Q = \{p_0\}$, and $\ell = p_0$. Hence $\ell = \max Q$, and Q covers $P_1 = \{p_0\}$ and is d -separated.

Assume now that the invariant holds at the beginning of an iteration $r \in [1 : n]$. If $p_r < \ell + d$, then ℓ covers p_r . Indeed, since $\ell \in Q \subseteq P_r$, we have $\ell \leq p_r$, and therefore $\ell \leq p_r < \ell + d$. In this case, the set Q remains unchanged. Thus Q covers P_{r+1} and remains d -separated. Moreover, $\ell = \max Q$ is still true.

Suppose now that $p_r \geq \ell + d$. Let Q' be the new value of Q , that is, $Q' = Q \cup \{p_r\}$, and let ℓ' be the new value of ℓ , that is, $\ell' = p_r$. Then Q' covers P_{r+1} , since Q covers P_r and p_r covers itself. Moreover, if $q \in Q$, then $q \leq \ell$, because $\ell = \max Q$. Hence $p_r - q \geq p_r - \ell \geq d$. Since Q is d -separated, the set Q' is also d -separated. Finally, $\ell' = p_r = \max Q'$.

Thus the invariant is preserved. When the algorithm stops, we have $P_n = P$. Hence $Q \subseteq P$ is a covering of P and is d -separated. Applying Lemma 2.2 with $Q^* := Q$ and $S^* := Q$, we conclude that Q is both a minimum covering of P and a maximum d -separated subset of P . $\square$

2.1.5 An implementation

We now present a simple $O(n \lg n)$ implementation of Algorithm 2.5 in C++, where n is the number of elements of P . For this implementation, and also for other algorithms yet to come, we will use *concepts*.

A *concept*² is a compile-time constraint on template arguments. It specifies syntactic requirements that a type must satisfy in order to be used in a given

²A concept in C++ plays a role similar to type classes in Haskell. Both restrict the types that can be used in polymorphic code. The analogy is partial, however. In C++, a concept is a compile-time

template. For instance, the concept `std::totally_ordered<T>` requires that objects of type `T` can be compared using the usual comparison operators, such as `<`, `<=`, `>`, `>=`, `==`, and `!=`.

The concept `std::sortable<I>` means that a sequence accessed through iterators of type `I` can be sorted by the standard sorting algorithms. In particular, the elements can be permuted and compared in the way required by sorting.

In the code below, the concept `additive_sortable<T>` requires `T` to be totally ordered, requires a vector of elements of type `T` to be sortable, and requires the expression `a + b` to be well-formed for objects `a` and `b` of type `T`, with result convertible to `T`.

```

1 template <typename T>
2 concept additive_sortable =
3     std::totally_ordered<T> &&
4     std::sortable<typename std::vector<T>::iterator> &&
5     requires(T a, T b) {
6         { a + b } -> std::convertible_to<T>;
7     };
8
9 template <additive_sortable T>
10 std::vector<T> interval_cover(std::vector<T> ps, T d) {
11     if (ps.empty()) return {};
12     std::sort(ps.begin(), ps.end());
13
14     std::vector<T> opt;
15     opt.reserve(ps.size());
16     auto i = ps.begin();
17     auto q = *i;
18     opt.push_back(q);
19     for (++i; i != ps.end(); ++i) {
20         if (*i >= q + d) {
21             q = *i;
22             opt.push_back(q);
23         }
24     }
25     return opt;
26 }
```

predicate on types, usually expressed by checking whether certain expressions are well-formed. In contrast, in Haskell, membership in a type class is made explicit by an instance declaration.

2.2 The task scheduling problem

2.2.1 The problem

Task structure A *task structure* is a triple (T, s, f) such that T is a finite set, s and f are real-valued functions whose domains contain T , that is, $s, t : T \rightsquigarrow \mathbb{R}$, and $s_i < f_i$ for every $i \in T$. The elements of T are called *tasks*. If $t \in T$, then we write s_t and f_t for $s(t)$ and $f(t)$, respectively, when convenient.

If (T, s, f) is a task structure and $X \subseteq T$, then (X, s, f) is also a task structure, called the *substructure induced by X* . Thus, when passing to a subproblem, we keep the same functions s and f and only replace the set of tasks.

Let (T, s, f) be a task structure. For distinct tasks i and k , we say that i and k are³ *compatible* if $[s_i, f_i) \cap [s_k, f_k) = \emptyset$; otherwise, they are *incompatible*.

A subset $I \subseteq T$ is *compatible* if every two distinct tasks in I are compatible. A compatible subset B of T is called *optimal in (T, s, f)* , or a *maximum compatible subset of (T, s, f)* , if no compatible subset of T has size greater than $|B|$. For brevity, we will simply say that such a set is *optimal in T* , or a *maximum compatible subset of T* , since the functions s and f are part of the ambient task structure.

Exercise 2.1

Let (T, s, f) be a task structure, and let $X \subseteq T$ be compatible. Show that there exists an ordering $x_0, \dots, x_{m-1}$ of the elements of X such that

$$f_{x_r} \leq s_{x_{r+1}} \quad \text{for every } r \in [0 : m - 1).$$

Exercise 2.2

Let (T, s, f) be a task structure. Write an algorithm that receives a subset $X \subseteq T$ and decides whether X is compatible. The running time of the algorithm must be $O(|X| \lg |X|)$.

The *task scheduling problem* consists in finding a compatible subset of T that is optimal in T , that is, a maximum compatible subset of T .

2.2.2 A recursive algorithm

General greedy scheme We will again explore the idea of a greedy algorithm, now for the task scheduling problem. The general scheme is the following. Given a task structure (T, s, f) with $T \neq \emptyset$, we first make a *greedy choice* according to some criterion, thereby selecting a task, say i . We then form the subproblem consisting of all remaining tasks that are compatible with i . After recursively

³For each $r, s \in \mathbb{R}$, $[r, s) := \{x \in \mathbb{R} \mid r \leq x < s\}$.

computing an optimal compatible subset for this subproblem, we try to extend it by adding i . Thus, in order to justify the algorithm, we must prove that the solution obtained in this way is optimal for the original task structure (T, s, f) .

Let us see that some natural greedy choices do not work.

Consider first the greedy choice that picks a task i that begins earliest, that is,

$$s_i = \min\{s_k \mid k \in T\}.$$

The following example shows that this strategy fails.

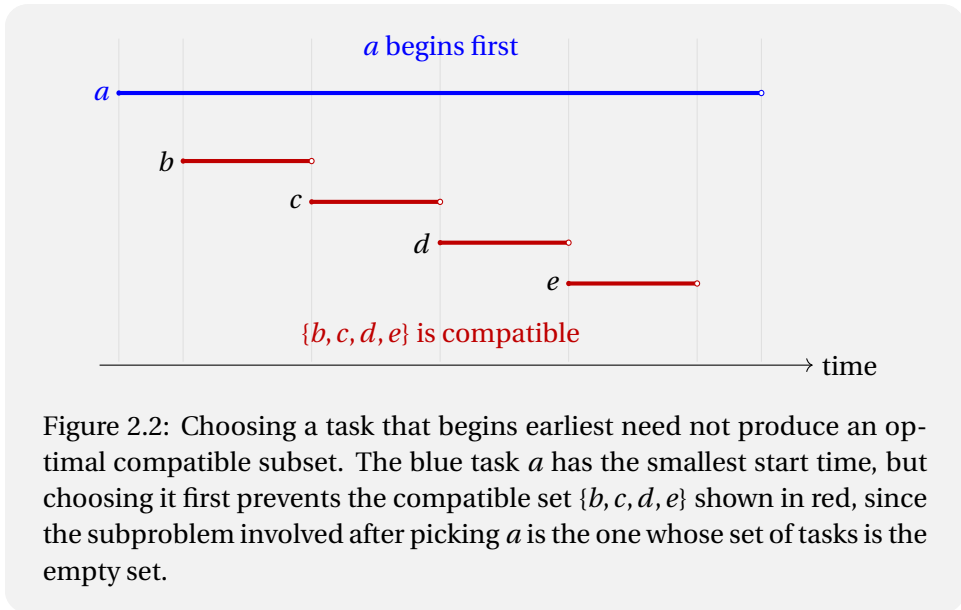


Figure 2.2: Choosing a task that begins earliest need not produce an optimal compatible subset. The blue task a has the smallest start time, but choosing it first prevents the compatible set $\{b, c, d, e\}$ shown in red, since the subproblem involved after picking a is the one whose set of tasks is the empty set.

Now consider the greedy choice that picks a task i that conflicts with a minimum number of other tasks, that is, such that the set

$$\{k \in T \setminus \{i\} \mid i \text{ and } k \text{ are incompatible}\}$$

has minimum size. The next example shows that this strategy does not work.

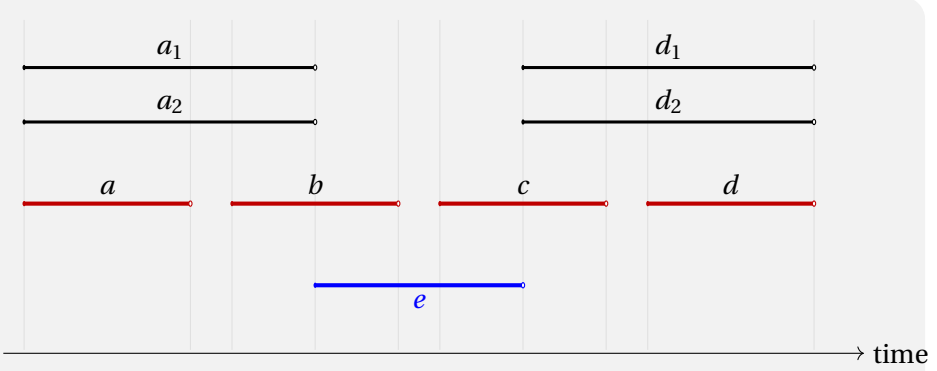


Figure 2.3: Choosing a task with a minimum number of conflicts need not produce an optimal compatible subset. The task e has only two conflicts, with b and c , but choosing it first prevents the compatible set $\{a, b, c, d\}$ shown in red.

Indeed, after the choice of e , the subproblem consists of the set of tasks

$$T' := \{a, a_1, a_2, d, d_1, d_2\}.$$

Every compatible subset of T' has size at most 2, since at most one task can be chosen from $\{a, a_1, a_2\}$, and at most one task can be chosen from $\{d, d_1, d_2\}$. Thus any solution obtained after choosing e has size at most 3, whereas $\{a, b, c, d\}$ is compatible and has size 4.

The greedy choice that actually works consists in choosing a task that finishes earliest, as we now show.

Before we state the theorem that justifies this greedy choice, and hence the recursive algorithm, let us make a preliminary observation. If (T, s, f) is a task structure and $X \subseteq T$, then (X, s, f) is also a task structure. Thus we shall often identify X with the induced task structure (X, s, f) ; in particular, expressions such as “optimal in X ” are always understood with respect to the ambient functions s and f .

Theorem 2.5. *Let (T, s, f) be a task structure with $T \neq \emptyset$. Let $i \in T$ be such that $f_i = \min\{f_t \mid t \in T\}$. Moreover, let*

$$T' := \{k \in T \mid s_k \geq f_i\}.$$

If B' is optimal in T' , then $B' \cup \{i\}$ is optimal in T .

Proof. Suppose that B' is optimal in T' . The set $B := B' \cup \{i\}$ is compatible, since every task k in B' satisfies $s_k \geq f_i$.

Now let J be any compatible subset of T . We show that $|J| \leq |B|$. If $J = \emptyset$, this is clear. Assume, therefore, that $J \neq \emptyset$, and let $j \in J$ be such that $f_j = \min\{f_t \mid$

$t \in J$. By the choice of i , we have $f_j \geq f_i$. Moreover, if $t \in J \setminus \{j\}$, then, since J is compatible and f_j is minimum among the finishing times of the tasks in J , we have $s_t \geq f_j \geq f_i$. Hence $J \setminus \{j\} \subseteq T'$.

Since $J \setminus \{j\}$ is compatible and B' is optimal in T' , we have $|J \setminus \{j\}| \leq |B'|$. Therefore $|J| \leq |B'| + 1 = |B' \cup \{i\}| = |B|$. Thus no compatible subset of T has size greater than $|B|$. Since B is compatible, it is optimal in T . $\square$

The theorem above immediately suggests the following recursive algorithm.

Algorithm 2.6 SCHEDULE(T, s, f)

Require: A finite set of tasks T

Ensure: A compatible subset of T that is optimal in T

```

1: if  $T = \emptyset$  then
2:   return  $\emptyset$ 
3: choose  $i \in T$  such that  $f_i = \min\{f_t \mid t \in T\}$ 
4:  $T' \leftarrow \{k \in T \mid s_k \geq f_i\}$ 
5:  $B' \leftarrow \text{SCHEDULE}(T', s, f)$ 
6: return  $B' \cup \{i\}$ 

```

Proof of correctness. We prove that, the call SCHEDULE(T, s, f) returns a compatible subset of T that is optimal in T for every task structure (T, s, f) . The proof is by induction on $|T|$.

If $T = \emptyset$, then the algorithm returns $\emptyset$, which is clearly optimal in T . Assume, then, that $T \neq \emptyset$, and let i and T' be the task and the set computed by the algorithm. Then $f_i = \min\{f_t \mid t \in T\}$ and

$$T' = \{k \in T \mid s_k \geq f_i\}.$$

Since $s_i < f_i$, we have $i \notin T'$, and hence $|T'| < |T|$. Therefore, by the induction hypothesis, the recursive call SCHEDULE(T', s, f) returns a subset B' of T' that is optimal in T' . By Theorem 2.5, the set $B' \cup \{i\}$ is optimal in T . Since this is precisely the set returned by the algorithm, the result follows. $\square$

Exercise 2.3

Let (T, s, f) be a task structure. Suppose that T admits a partition $\{T_1, T_2\}$ such that every task in T_1 is compatible with every task in T_2 . Let I_1 be a compatible subset of T_1 that is maximum in T_1 , and let I_2 be a compatible subset of T_2 that is maximum in T_2 . Show that $I_1 \cup I_2$ is a compatible subset of T that is maximum in T .

2.2.3 An iterative approach

Let $\mathcal{J}$ be the collection of all compatible subsets of a task structure (T, s, f) . Note that, if $J \in \mathcal{J}$ and $I \subseteq J$, then $I \in \mathcal{J}$. This is a simple property, but it is worth recording.

Let $X \subseteq T$. We say that $I \subseteq X$ is ***X-good*** if there exists a compatible subset B of T that is optimal in T and such that

$$I \subseteq B \quad \text{and} \quad B \setminus I \subseteq T \setminus X.$$

In other words, I can be extended to an optimal compatible subset B of T that uses no element of $X \setminus I$.

Theorem 2.6. *Let (T, s, f) be a task structure, and let $X \subset T$. Suppose that $I \subseteq X$ is X -good. Let $j \in T \setminus X$ be such that*

$$f_j = \min\{f_t \mid t \in T \setminus X\}.$$

Then:

- (i) *if $I \cup \{j\} \notin \mathcal{J}$, then I is $(X \cup \{j\})$ -good;*
- (ii) *if $I \cup \{j\} \in \mathcal{J}$, then $I \cup \{j\}$ is $(X \cup \{j\})$ -good.*

Proof. Since I is X -good, there exists a compatible subset B of T that is optimal in T and such that

$$I \subseteq B \quad \text{and} \quad B \setminus I \subseteq T \setminus X.$$

For the proof of (i), suppose that $I \cup \{j\} \notin \mathcal{J}$. Then $j \notin B$, since otherwise $I \cup \{j\}$ would be a compatible subset of B . Hence $B \setminus I \subseteq T \setminus (X \cup \{j\})$. Therefore I is $(X \cup \{j\})$ -good.

For the proof of (ii), suppose that $I' := I \cup \{j\} \in \mathcal{J}$, and let $X' := X \cup \{j\}$. If $j \in B$, then $I' \subseteq B$ and $B \setminus I' \subseteq T \setminus X'$. Thus I' is X' -good.

Suppose, then, that $j \notin B$. Since B is optimal in T , the set $B \cup \{j\}$ is not compatible. Hence there exists $i \in B$ such that i and j are incompatible. Since $I' = I \cup \{j\}$ is compatible, the task j is compatible with every task in I . Thus $i \notin I$. Therefore $i \in B \setminus I \subseteq T \setminus X$, and by the choice of j we have $f_j \leq f_i$.

We claim that $B' := (B \setminus \{i\}) \cup \{j\}$ is compatible. It is enough to show that j is compatible with every task in $B \setminus \{i\}$. This is clear for the tasks in I , since $I \cup \{j\}$ is compatible.

Now let $t \in B \setminus (I \cup \{i\})$. Since $B \setminus I \subseteq T \setminus X$, we have $t \in T \setminus X$. By the choice of j , it follows that $f_j \leq f_t$. We also have $f_j \leq f_i$, as observed above.

Since i and t are compatible, either $s_i \geq f_t$ or $s_t \geq f_i$. The first alternative is impossible: if $s_i \geq f_t$, then $s_i \geq f_t \geq f_j$, and hence i and j would be compatible, a contradiction. Therefore $s_t \geq f_i$. Since $f_i \geq f_j$, we get $s_t \geq f_j$, and so t and j are compatible. This proves that $B' \in \mathcal{J}$.

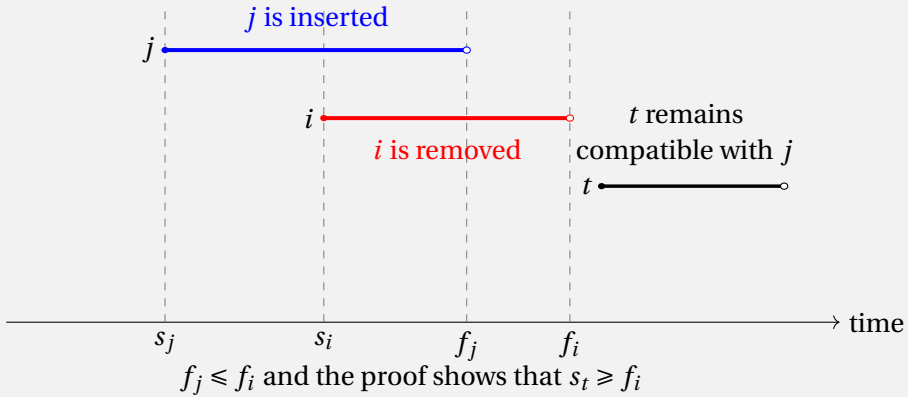


Figure 2.4: The exchange step in the proof of part (ii). The task j replaces a conflicting task i . The remaining tasks $t \in B \setminus (I \cup \{i\})$ are shown to start no earlier than f_i , and hence are compatible with j .

Since $|B'| = |B|$, the set B' is also optimal in T . Moreover, $I' \subseteq B'$. Finally, we show that $B' \setminus I' \subseteq T \setminus X'$. Indeed, let $k \in B' \setminus I'$. Since $B' = (B \setminus \{i\}) \cup \{j\}$ and $I' = I \cup \{j\}$, we have $k \in B \setminus \{i\}$, $k \notin I$, and $k \neq j$; hence $k \in B \setminus I$. But $B \setminus I \subseteq T \setminus X$, so $k \in T \setminus X$. Since $X' = X \cup \{j\}$ and $k \neq j$, it follows that $k \in T \setminus X'$. Hence I' is X' -good, as required. $\square$

The preceding theorem suggests the following iterative greedy algorithm.

Algorithm 2.7 GREEDYSCHEDULE(T, s, f)

Require: A task structure (T, s, f)

Ensure: A compatible subset of T that is optimal in T

```

1:  $X \leftarrow \emptyset$ 
2:  $I \leftarrow \emptyset$ 
3: while  $X \neq T$  do
4:   choose  $j \in T \setminus X$  such that  $f_j = \min\{f_t \mid t \in T \setminus X\}$ 
5:   if  $I \cup \{j\}$  is compatible then
6:      $I \leftarrow I \cup \{j\}$ 
7:    $X \leftarrow X \cup \{j\}$ 
8: return  $I$ 

```

At the beginning of each iteration, I is X -good. This is true at the beginning of the first iteration, because $\emptyset$ is $\emptyset$ -good. Now assume that I is X -good at the beginning of an iteration and that $X \neq T$. The theorem shows that, after the

next task j is processed, the updated set I is $(X \cup \{j\})$ -good. Thus I is X -good throughout all iterations of the algorithm.

When the algorithm stops, we have $X = T$, and therefore I is T -good. Hence there exists a compatible subset B of T that is optimal in T and such that $I \subseteq B$ and $B \setminus I \subseteq T \setminus T$. Thus $B \setminus I = \emptyset$, and so $B \subseteq I$. Therefore $B = I$, and hence I is optimal in T . $\square$

It is easy to obtain from this version the usual $O(n \lg n)$ implementation of the greedy algorithm, where $n := |T|$. In this implementation, instead of explicitly maintaining the set X of processed tasks, we order the tasks as $t_0, \dots, t_{n-1}$ so that $f(t_0) \leq \dots \leq f(t_{n-1})$, and scan them in this order.

At the beginning of each iteration, let X be the set $\{t_q \mid 0 \leq q < r\}$. Hence, by the invariant proved above, I is X -good throughout all iterations. Moreover, to test whether t_r is compatible with all tasks already chosen, it is enough to keep track of the last chosen task ℓ .

Algorithm 2.8 GREEDYSCHEDULEFAST(T, s, f)

Require: A task structure (T, s, f) with $T \neq \emptyset$

Ensure: A compatible subset of T that is optimal in T

```

1: let  $t_0, \dots, t_{n-1}$  be an ordering of  $T$  such that  $f(t_0) \leq \dots \leq f(t_{n-1})$ 
2:  $I \leftarrow \{t_0\}$ 
3:  $\ell \leftarrow t_0$ 
4: for  $r \leftarrow 1$  to  $n - 1$  do
5:   if  $s(t_r) \geq f(\ell)$  then
6:      $I \leftarrow I \cup \{t_r\}$ 
7:      $\ell \leftarrow t_r$ 
8: return  $I$ 
```

Complexity analysis Let $n := |T|$. The ordering of the tasks as $t_0, \dots, t_{n-1}$ such that $f(t_0) \leq \dots \leq f(t_{n-1})$ can be computed in $O(n \lg n)$ time. After that, all initializations take $O(1)$ time. Each iteration of the algorithm can be implemented in $O(1)$ time. Since there are $n - 1$ iterations, a complete execution of the loop takes $O(n)$ time. Therefore, the total running time of the algorithm is $O(n \lg n)$.

2.2.4 An implementation

Here is an implementation of Algorithm 2.8. The code is followed by a few remarks.

```

1 template <typename T>
2 concept task_time =
3     std::totally_ordered<T> &&
4     std::copyable<T>;
5
```



```

6  template <task_time T>
7  struct task {
8      T start;
9      T finish;
10
11     task() = delete;
12
13     task(const T& s, const T& f)
14         : start{s}, finish{f} {}
15
16     bool precedes(const task& other) const {
17         return finish <= other.start;
18     }
19 };
20
21 template <task_time T>
22 struct by_finish {
23     bool operator()(const task<T>& x, const task<T>& y) const {
24         return x.finish < y.finish;
25     }
26 };
27
28 template <typename T>
29 concept sortable_tasks_by_finish =
30     task_time<T> &&
31     std::sortable<
32         typename std::vector<task<T>>::iterator,
33         by_finish<T>
34     >;
35
36 template <typename T>
37 requires sortable_tasks_by_finish<T>
38 std::vector<task<T>> fast_sched(std::vector<task<T>> ts) {
39     if (ts.empty()) return {};
40     std::sort(ts.begin(), ts.end(), by_finish<T>{});
41     std::vector<task<T>> opt;
42     opt.reserve(ts.size());
43     auto last = ts.begin();
44     opt.push_back(*last);
45     for (auto i = std::next(ts.begin()); i != ts.end(); ++i) {
46         if (last->precedes(*i)) {
47             opt.push_back(*i);
48             last = i;
49         }
50     }
51     return opt;
52 }

```

The concept `task_time<T>` collects the requirements imposed on the type used to represent times. The condition `std::totally_ordered<T>` is needed because we compare starting and finishing times, for instance in the expression `finish <= other.start`, and because we sort tasks by finishing time. The condition `std::copyable<T>` is needed because objects of type `T` are copied when tasks are constructed.

2.2.5 A min-max approach

Let (T, s, f) be a task structure. A *clique* is a subset C of T such that, for all $i, k \in C$ with $i \neq k$, we have $[s_i, f_i) \cap [s_k, f_k) \neq \emptyset$.⁴

Notice that, if C is a nonempty clique, then

$$\bigcap_{t \in C} [s_t, f_t) \neq \emptyset.$$

Indeed, suppose that C is a nonempty clique, and let $m \in C$ be such that $s_m = \max\{s_t \mid t \in C\}$. We show that $s_m \in [s_t, f_t)$ for every $t \in C$. Let $t \in C$. By the choice of m , we have $s_t \leq s_m$. Also, $s_m < f_t$; this is immediate if $t = m$, and follows from the fact that C is a clique otherwise. Hence $s_t \leq s_m < f_t$, and therefore $s_m \in [s_t, f_t)$. Thus

$$\bigcap_{t \in C} [s_t, f_t) \neq \emptyset.$$

Moreover, it follows that

(2.1) if I is compatible and C is a clique, then $|C \cap I| \leq 1$.

Exercise 2.4

Let (T, s, f) be a task structure, and let $C \subseteq T$. Write an algorithm that decides whether C is a clique in time $O(|C|)$.

Before we turn to the next concept, let us recall the definition of a partition. Given a set U , we say that a collection⁵ $\mathcal{C}$ of nonempty subsets of U is a *partition* of U if

- (i) $C_1 \cap C_2 = \emptyset$ for every $C_1, C_2 \in \mathcal{C}$ with $C_1 \neq C_2$, and
- (ii) $\bigcup \mathcal{C} = U$.

A *clique-partition* of T is a partition $\mathcal{C}$ of T such that each set in $\mathcal{C}$ is a clique. Thus, Assertion (2.1) implies that

$$\min\{|\mathcal{C}| : \mathcal{C} \text{ is a clique-partition of } T\} \geq \max\{|I| : I \text{ is a compatible subset of } T\}.$$

⁴This is a special case of the so called *Helly property* for intervals on the real line. A finite family of intervals is pairwise intersecting if and only if it has a common point.

⁵If $\mathcal{C}$ is a collection of sets, then $\bigcup \mathcal{C} := \{x \mid x \in C \text{ for some } C \in \mathcal{C}\}$.

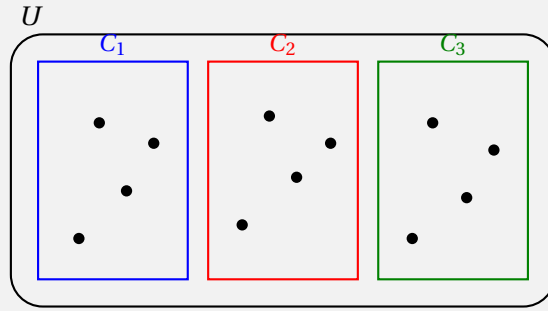


Figure 2.5: A partition of a set U into three nonempty, pairwise disjoint parts.

Indeed, assume that $\mathcal{C}$ is a clique-partition of T and that I is a compatible subset of T . Then

$$|I| = \left| \bigcup_{C \in \mathcal{C}} (I \cap C) \right| = \sum_{C \in \mathcal{C}} |I \cap C| \leq \sum_{C \in \mathcal{C}} 1 = |\mathcal{C}|.$$

In order to appreciate this concept, let us introduce the notion of the *conflict graph* of T . This is the graph whose vertex set is T and whose edge set consists of all pairs $\{u, v\}$ such that $u, v \in T$, $u \neq v$, and u and v are incompatible. The next figure illustrates this idea.

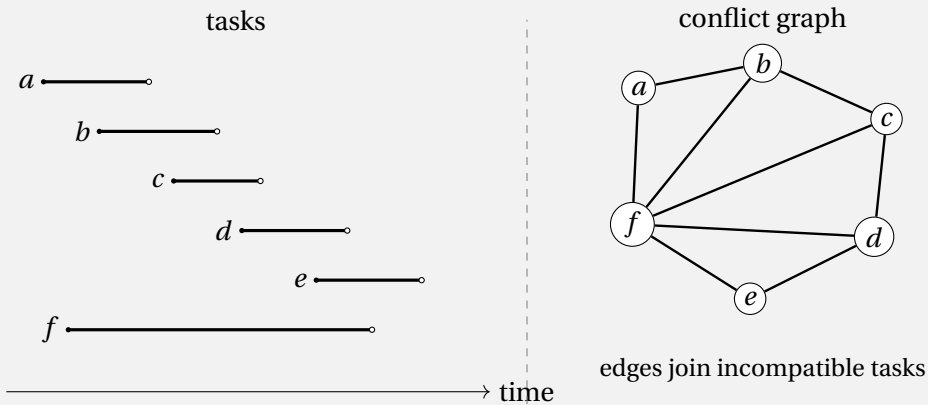


Figure 2.6: Tasks represented as intervals and their conflict graph. Two vertices are adjacent exactly when the corresponding tasks are incompatible.

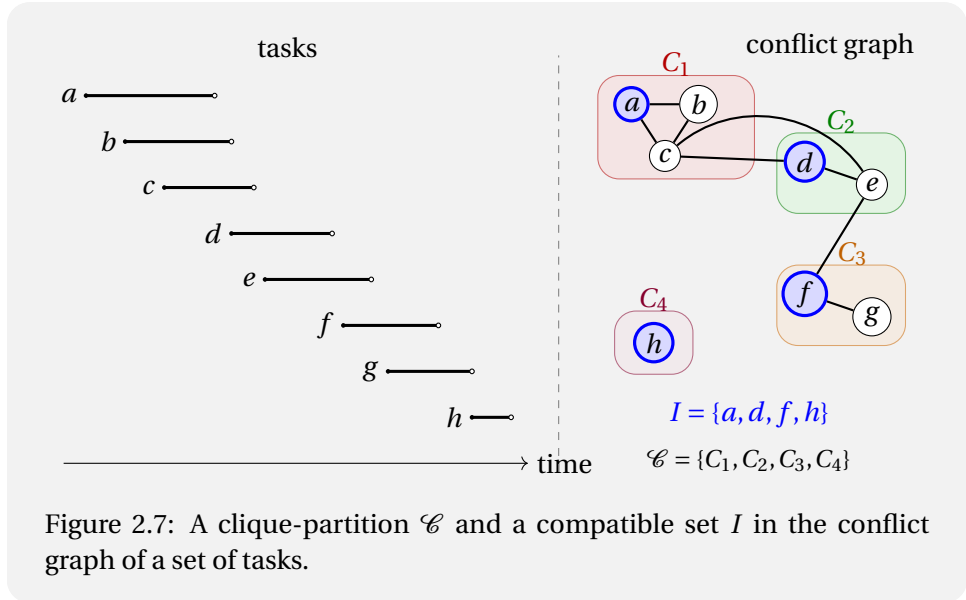
The next theorem establishes that equality holds. Although it requires a bit more preparation, the proof is smoother.

Theorem 2.7. *For each task structure (T, s, f) , there exists a clique-partition $\mathcal{C}$ of T and a compatible subset I of T such that $|\mathcal{C}| = |I|$.*

Proof. We prove the theorem by induction on $|T|$. Let (T, s, f) be a task structure. The result is clear if $T = \emptyset$. Suppose, therefore, that $T \neq \emptyset$. Let $i \in T$ be such that $f_i = \min\{f_j \mid j \in T\}$, and set $C := \{j \in T \mid s_j < f_i\}$. Note that C is nonempty, since $i \in C$. We show that C is a clique. Let $j \in C$ be such that $s_j = \max\{s_t \mid t \in C\}$. If $t \in C$, then $s_t \leq s_j < f_i \leq f_t$. Hence s_j is a point in $[s_t, f_t]$ for every $t \in C$. Therefore C is a clique.

Set $T' := T \setminus C$. By the induction hypothesis, there exists a clique-partition $\mathcal{C}'$ of T' and a compatible subset I' of T' such that $|\mathcal{C}'| = |I'|$. It is clear that $\mathcal{C} := \mathcal{C}' \cup \{C\}$ is a clique-partition of T . Moreover, $I := I' \cup \{i\}$ is a compatible subset of T , since every task in T' starts no earlier than f_i . Finally, since $i \notin T'$, we have $|I| = |I'| + 1$, and since $C \notin \mathcal{C}'$, we have $|\mathcal{C}| = |\mathcal{C}'| + 1$. Thus $|\mathcal{C}| = |I|$, and the result follows. $\square$

It is worth pointing out that the following is an NP-hard problem.⁶ Given a graph G , find a partition of minimum size of its vertex set into cliques. Notice that, if G is the conflict graph of a task structure, then Theorem 2.7 shows that this restricted problem lives in P, which means that it can be solved in polynomial time.



⁶Although you may not yet know what this means, in practice it means that no polynomial-time algorithm is known for this problem.

Algorithm 2.9 CLIQUEPARTITION(T, s, f)**Require:** A task structure (T, s, f) **Ensure:** A pair $(\mathcal{C}, I)$ where $\mathcal{C}$ is a clique-partition of T , and $I \subseteq T$ is compatible with $|\mathcal{C}| = |I|$

```

1: if  $T = \emptyset$  then
2:   return  $(\emptyset, \emptyset)$ 
3: choose  $i \in T$  such that  $f_i = \min\{f_t \mid t \in T\}$ 
4:  $C \leftarrow \{j \in T \mid s_j < f_i\}$ 
5:  $T' \leftarrow T \setminus C$ 
6:  $(\mathcal{C}', I') \leftarrow \text{CLIQUEPARTITION}(T', s, f)$ 
7:  $\mathcal{C} \leftarrow \mathcal{C}' \cup \{C\}$ 
8:  $I \leftarrow I' \cup \{i\}$ 
9: return  $(\mathcal{C}, I)$ 

```

It is a simple matter to derive an iterative version from the recursive one.

Algorithm 2.10 CLIQUEPARTITIONITERATIVE(T, s, f)**Require:** A task structure T **Ensure:** A pair $(\mathcal{C}, I)$ where $\mathcal{C}$ is a clique-partition of T , and $I \subseteq T$ is compatible with $|\mathcal{C}| = |I|$

```

1:  $R \leftarrow T$ 
2:  $\mathcal{C} \leftarrow \emptyset$ 
3:  $I \leftarrow \emptyset$ 
4: while  $R \neq \emptyset$  do
5:   choose  $i \in R$  such that  $f_i = \min\{f_t \mid t \in R\}$ 
6:    $C \leftarrow \{j \in R \mid s_j < f_i\}$ 
7:    $\mathcal{C} \leftarrow \mathcal{C} \cup \{C\}$ 
8:    $I \leftarrow I \cup \{i\}$ 
9:    $R \leftarrow R \setminus C$ 
10: return  $(\mathcal{C}, I)$ 

```

The following is an $O(n \lg n)$ -time algorithm for constructing a clique-partition $\mathcal{C}$ of T and a compatible subset $I \subseteq T$ such that $|\mathcal{C}| = |I|$, where $n := |T|$. Some observations are in order. For the formal object $\perp$, we assume that $f(\perp) = -\infty$.

The construction of the compatible set I is straightforward. It is the same construction used in Algorithm 2.8.

At the beginning of each iteration of the outer loop, the following invariant holds:

- (i) $\mathcal{C}$ is a clique-partition of $\{u_k \mid k \in [0 : j]\}$;
- (ii) I is compatible;
- (iii) $|\mathcal{C}| = |I|$;

(iv) $s(u_k) < f(\ell)$ for every $k \in [0 : j)$, and, if $j < n$, then $s(u_j) \geq f(\ell)$.

Exercise 2.5

Prove the validity of this invariant.

Algorithm 2.11 CLIQUEPARTITIONFAST(T, s, f)

Require: A task structure (T, s, f)

Ensure: A pair $(\mathcal{C}, I)$

where $\mathcal{C}$ is a clique-partition of T , and $I \subseteq T$ is compatible with $|\mathcal{C}| = |I|$

```

1: if  $T = \emptyset$  then
2:   return  $(\emptyset, \emptyset)$ 
3: let  $t_0, \dots, t_{n-1}$  be an ordering of  $T$  such that  $f(t_0) \leq \dots \leq f(t_{n-1})$ 
4: let  $u_0, \dots, u_{n-1}$  be an ordering of  $T$  such that  $s(u_0) \leq \dots \leq s(u_{n-1})$ 
5:  $\mathcal{C} \leftarrow \emptyset$ 
6:  $I \leftarrow \emptyset$ 
7:  $j \leftarrow 0$ 
8:  $\ell \leftarrow \perp$ 
9: for  $i \leftarrow 0$  to  $n - 1$  do
10:  if  $s(t_i) \geq f(\ell)$  then
11:     $I \leftarrow I \cup \{t_i\}$ 
12:     $\ell \leftarrow t_i$ 
13:     $C \leftarrow \emptyset$ 
14:    while  $j < n$  and  $s(u_j) < f(\ell)$  do
15:       $C \leftarrow C \cup \{u_j\}$ 
16:       $j \leftarrow j + 1$ 
17:     $\mathcal{C} \leftarrow \mathcal{C} \cup \{C\}$ 
18: return  $(\mathcal{C}, I)$ 
```

Exercise 2.6

Consider the proof of Theorem 2.7. Instead of choosing a task with minimum finishing time, choose a task $i \in T$ with maximum starting time, that is,

$$s_i = \max\{s_t \mid t \in T\}.$$

Use this choice to prove again that, for each task structure (T, s, f) , there exists a clique-partition $\mathcal{C}$ of T and a compatible subset I of T such that $|\mathcal{C}| = |I|$.

Exercise 2.7

Extract from the proof of Exercise 2.6 an algorithm that receives a task structure (T, s, f) and returns a clique-partition $\mathcal{C}$ of T and a compatible subset $I \subseteq T$ such that $|\mathcal{C}| = |I|$.

Exercise 2.8

Let (T, s, f) be a task structure. Suppose that there exists $i \in T$ such that

$$C := \{i\} \cup \{k \in T \setminus \{i\} \mid i \text{ and } k \text{ are incompatible}\}$$

is a clique. Use this assumption to prove that there exists a clique-partition $\mathcal{C}$ of T and a compatible subset I of T such that $|\mathcal{C}| = |I|$.

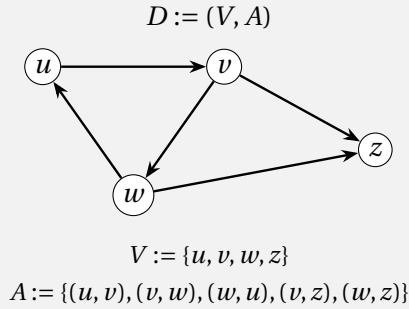
Notice that the two greedy choices discussed above satisfy this property: choosing a task with minimum finishing time, and choosing a task with maximum starting time.

Exercise 2.9

The proof of Theorem 2.7 may be carried out by choosing a task with minimum finishing time or by choosing a task with maximum starting time. Compare the two resulting recursive algorithms. Do they always produce the same clique-partition?

2.2.6 Longest path approach

The task scheduling problem can also be seen as a longest path problem in an acyclic digraph. Let us recall some definitions before we introduce the reduction. There are several competing definitions of the notion of a digraph, depending on the applications considered. For the purposes of this section, a *digraph* $D := (V, A)$ consists of a nonempty finite set V , called the *set of vertices* of D , and a set $A \subseteq V \times V$, called the *set of arcs* of D , such that $u \neq v$ for every $(u, v) \in A$. Thus each arc of D is an ordered pair of distinct vertices.

Figure 2.8: A digraph with vertex set V and arc set A .

In this text, $\mathbb{N}$ includes 0. For $r, s \in \mathbb{N}$, let

$$[r : s] := \{i \in \mathbb{N} \mid r \leq i \leq s\} \quad \text{and} \quad [r : s) := \{i \in \mathbb{N} \mid r \leq i < s\}.$$

Notice that $[r : s] = \emptyset$ whenever $r > s$.

Let $D := (V, A)$ be a digraph. For each $k \in \mathbb{N}$, a sequence $P := \langle v_i \mid i \in [0 : k] \rangle$ of vertices, which we also write as $\langle v_0, v_1, \dots, v_k \rangle$, is a **(v_0, v_k) -walk**, or simply a **walk**, of D if $(v_{i-1}, v_i) \in A$ for all $i \in [1 : k]$. The number k is called the **length** of the walk. The walk P is a **path** if its vertices are pairwise distinct, that is, if $v_i \neq v_j$ for all $i, j \in [0 : k]$ with $i \neq j$.

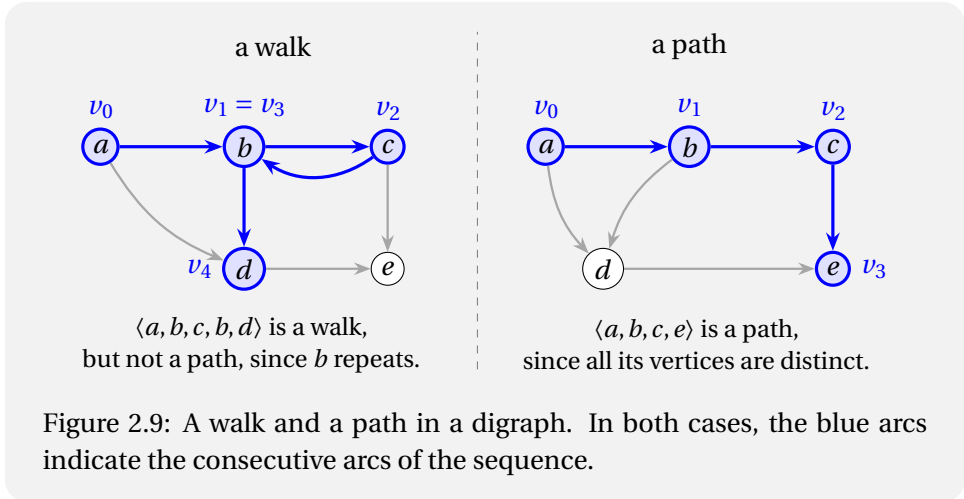
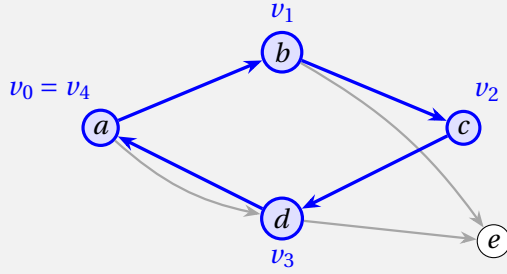


Figure 2.9: A walk and a path in a digraph. In both cases, the blue arcs indicate the consecutive arcs of the sequence.

A **cycle** in D is a walk $\langle v_i \mid i \in [0 : k] \rangle$ of length $k \geq 2$ such that $v_0 = v_k$ and $\langle v_i \mid i \in [0 : k] \rangle$ is a path.



$C = \langle a, b, c, d, a \rangle$ is a cycle: it is a closed walk,
and $\langle a, b, c, d \rangle$ is a path.

Figure 2.10: A cycle in a digraph. The blue arcs indicate the cycle $\langle a, b, c, d, a \rangle$.

Let (T, s, f) be a task structure. The *compatibility digraph* of T is the digraph $D_T = (T, A)$ whose arc set is

$$A := \{(u, v) \mid u, v \in T \text{ and } s_v \geq f_u\}.$$

Thus, there is an arc from u to v precisely when task v can be scheduled after task u .

Note that D_T is acyclic. Indeed, if $(u, v) \in A$, then $f_u \leq s_v < f_v$. Thus, for all $u, v \in T$, if there exists a (u, v) -path in D_T of positive length, then $f_u < f_v$. In particular, there cannot exist both a (u, v) -path and a (v, u) -path of positive length. Hence D_T contains no cycle.

Moreover, the set of vertices of every path in D_T is a compatible set of tasks. Indeed, if $\langle u_0, u_1, \dots, u_k \rangle$ is a path in D_T , then $s(u_r) \geq f(u_{r-1})$ for every $r \in [1 : k]$. Hence $s(u_q) \geq f(u_p)$ whenever $p < q$, and so $\{u_0, u_1, \dots, u_k\}$ is a compatible set.

Conversely, if I is a nonempty compatible set of tasks, then its elements can be ordered as $P := \langle u_0, \dots, u_k \rangle$ so that $f(u_0) \leq \dots \leq f(u_k)$. Since I is compatible, we have $s(u_r) \geq f(u_{r-1})$ for every $r \in [1 : k]$. Thus $(u_{r-1}, u_r) \in A$ for every $r \in [1 : k]$, whence P is a path in D_T .

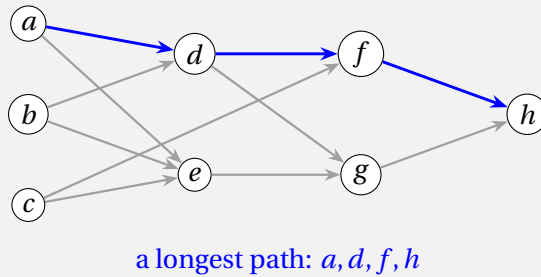
Therefore, the nonempty compatible subsets of T are exactly the vertex sets of paths in D_T . Hence, when $T \neq \emptyset$, an optimal compatible subset of T can be obtained by finding a longest path in the acyclic digraph D_T .

Note also that D_T is transitive. Indeed, if (u, v) and (v, w) are arcs of D_T , then $s_v \geq f_u$ and $s_w \geq f_v$. Since $s_v < f_v$, it follows that

$$f_u \leq s_v < f_v \leq s_w.$$

Hence $s_w \geq f_u$, and therefore (u, w) is an arc of D_T . Thus the compatibility digraph may contain many arcs that are forced by transitivity. For this reason, when drawing it, it is often clearer to draw only its transitive reduction.

A **transitive reduction** of a digraph $D = (V, A)$ is a digraph $D' = (V, A')$ such that $A' \subseteq A$ and A' is minimal, with respect to inclusion, among the subsets of A satisfying the following property: for all $u, v \in V$, if D contains a (u, v) -path, then D' also contains a (u, v) -path. Since $A' \subseteq A$, every path in D' is also a path in D . Thus the condition above says precisely that D and D' have the same reachability relation. When D is acyclic, its transitive reduction is unique.



The omitted arcs are obtained by transitivity.

Figure 2.11: The transitive reduction of the compatibility digraph associated with the tasks from the previous example. Its transitive closure is the full compatibility digraph.

The longest path problem in acyclic digraphs will be studied in a more general setting in a later chapter.

2.3 Scheduling all tasks

Now we turn to the problem of finding the minimum number of resources needed to allocate all tasks in such a way that the set of tasks assigned to each resource is compatible.

Let (T, s, f) be a task structure. Recall that a clique is a set of pairwise incompatible tasks. A partition $\mathcal{P}$ of T is called a **compatible partition** if every $P \in \mathcal{P}$ is compatible. We are now interested in finding a compatible partition $\mathcal{P}$ of T with as few parts as possible.

Assume that $C \subseteq T$ is a clique and that $\mathcal{P}$ is a compatible partition of T . Then

$$|C| = \left| \bigcup_{P \in \mathcal{P}} (C \cap P) \right| = \sum_{P \in \mathcal{P}} |C \cap P| \leq \sum_{P \in \mathcal{P}} 1 = |\mathcal{P}|.$$

Thus,

$$\min\{|\mathcal{P}| \mid \mathcal{P} \text{ is a compatible partition of } T\} \geq \max\{|C| \mid C \subseteq T \text{ is a clique}\}.$$

The next theorem shows that equality holds.

Theorem 2.8. *For each task structure (T, s, f) , there exists a compatible partition $\mathcal{P}$ of T and a clique $C \subseteq T$ such that $|\mathcal{P}| = |C|$.*

Proof. We prove the theorem by induction on $|T|$. Let T be a finite set of tasks. If $T = \emptyset$, the result is clear. Assume, therefore, that $T \neq \emptyset$. Let $i \in T$ be such that $s_i = \max\{s_t \mid t \in T\}$, and set $T' := T \setminus \{i\}$.

By the induction hypothesis, there exists a compatible partition $\mathcal{P}'$ of T' and a clique $C' \subseteq T'$ such that $|\mathcal{P}'| = |C'|$. We have two cases to consider.

First, suppose that there exists $Q \in \mathcal{P}'$ such that $Q \cup \{i\}$ is compatible. Let

$$\mathcal{P} := (\mathcal{P}' \setminus \{Q\}) \cup \{Q \cup \{i\}\}.$$

Then $\mathcal{P}$ is a compatible partition of T , and $|\mathcal{P}| = |\mathcal{P}'| = |C'|$. Thus $\mathcal{P}$ and C' establish the result.

Suppose now that, for every $Q \in \mathcal{P}'$, the set $Q \cup \{i\}$ is not compatible. Thus, for every $Q \in \mathcal{P}'$, there is at least one task in Q that is incompatible with i . We claim that there is exactly one such task.

Indeed, let $j \in Q$ be incompatible with i . By the choice of i , we have $s_j \leq s_i$, and, since i and j are incompatible, we have $s_i < f_j$. Hence $s_i \in [s_j, f_j)$. Now suppose that $j, k \in Q$ are both incompatible with i . Then

$$s_i \in [s_j, f_j) \cap [s_k, f_k).$$

Since Q is compatible, this implies that $j = k$.

Therefore, for each $Q \in \mathcal{P}'$, there is a unique element of Q that is incompatible with i ; denote it by j_Q . Let

$$C := \{i\} \cup \{j_Q \mid Q \in \mathcal{P}'\}.$$

The set C is a clique: indeed, the point s_i belongs to the orresponding to i , and, by the previous observation, it also belongs to the interval corresponding to every task j_Q .

Now let $\mathcal{P} := \mathcal{P}' \cup \{\{i\}\}$. It is clear that $\mathcal{P}$ is a compatible partition of T . Moreover, the elements j_Q , with $Q \in \mathcal{P}'$, are distinct, since the sets in $\mathcal{P}'$ are pairwise disjoint. Hence $|C| = |\mathcal{P}'| + 1$. Therefore $|\mathcal{P}| = |\mathcal{P}'| + 1 = |C|$, as required. $\square$

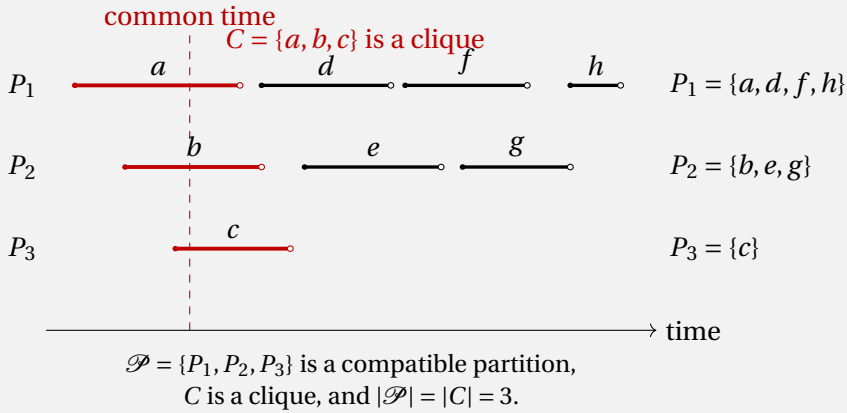
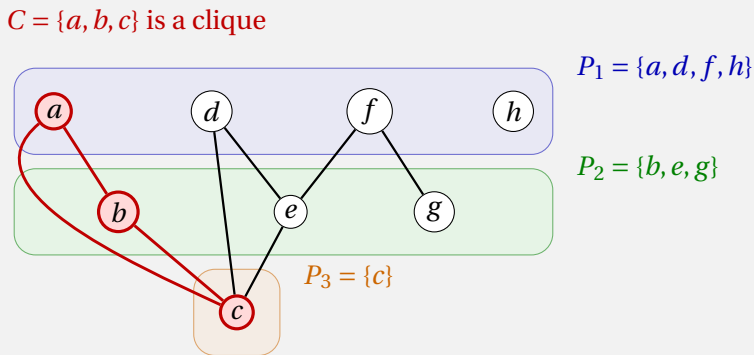


Figure 2.12: A compatible partition and a clique of the same size. The three red intervals overlap at a common time, so they form a clique. Each row is a compatible set of tasks, and the rows form a partition of all tasks.



$\mathcal{P} = \{P_1, P_2, P_3\}$ is a compatible partition: each P_i is a stable set in the conflict graph, and $|C| = |\mathcal{P}| = 3$.

Figure 2.13: The conflict graph associated with the tasks in Figure 2.12. The shaded regions form a compatible partition, and the red vertices form a clique.

Algorithm 2.12 COMPATIBLEPARTITION(T, s, f)**Require:** A task structure (T, s, f) **Ensure:** A pair $(\mathcal{P}, C)$ such that $\mathcal{P}$ is a compatible partition of T , $C \subseteq T$ is a clique, and $|\mathcal{P}| = |C|$

```

1: if  $T = \emptyset$  then
2:   return  $(\emptyset, \emptyset)$ 
3: choose  $i \in T$  such that  $s_i = \max\{s_t \mid t \in T\}$ 
4:  $T' \leftarrow T \setminus \{i\}$ 
5:  $(\mathcal{P}', C') \leftarrow \text{COMPATIBLEPARTITION}(T', s, f)$ 
6: for each  $Q \in \mathcal{P}'$  do
7:   if  $Q \cup \{i\}$  is compatible then
8:      $\mathcal{P} \leftarrow (\mathcal{P}' \setminus \{Q\}) \cup \{Q \cup \{i\}\}$ 
9:   return  $(\mathcal{P}, C')$ 
10:  $C \leftarrow \{i\}$ 
11: for each  $Q \in \mathcal{P}'$  do
12:   let  $j_Q$  be the unique task in  $Q$  that is incompatible with  $i$ 
13:    $C \leftarrow C \cup \{j_Q\}$ 
14:  $\mathcal{P} \leftarrow \mathcal{P}' \cup \{\{i\}\}$ 
15: return  $(\mathcal{P}, C)$ 

```

Exercise 2.10

Consider the proof of Theorem 2.8. Suppose that, for every $Q \in \mathcal{P}'$, the set $Q \cup \{i\}$ is not compatible. Show that, for each $Q \in \mathcal{P}'$, the task j_Q , defined in the proof, satisfies

$$s_{j_Q} = \max\{s_t \mid t \in Q\}.$$

Conclude also that

$$f_{j_Q} = \max\{f_t \mid t \in Q\}.$$

Using the result of this preceding exercise, we can write a more efficient version of the algorithm. The next algorithm takes $O(n \lg n)$ time, where $n := |T|$.

Recall the definition of a priority queue. Let V be a set, whose elements are called *values*, and let K be a totally ordered set, whose elements are called *keys*. A *priority queue* with values in V and keys in K is a multiset of pairs $(v, k) \in V \times K$. For a priority queue H , the operation $\min(H)$, when $H \neq \emptyset$, returns the minimum key of a pair in H . The operation $\text{remove}(H)$, when $H \neq \emptyset$, removes and returns a pair $(v, k) \in H$ of minimum key. Finally, the operation $\text{insert}(H, (v, k))$ inserts the pair (v, k) into H .

The algorithm maintains a priority queue H with values in $\mathcal{P}(T)$ and keys in

$\mathbb{R}$. So, objects of H are pairs (P, r) , where $P \subseteq T$ and $r \in \mathbb{R}$ is the *key* of the pair. In the algorithm below, P is a compatible set of tasks, and r is the finishing time of the last task inserted into P . The variable k stores the task that opened the most recent new part of the partition. At the end, the tasks whose intervals contain s_k form the clique certificate.

Algorithm 2.13 COMPATIBLEPARTITIONFAST(T, s, f)

Require: A task structure (T, s, f) with $T \neq \emptyset$

Ensure: A pair $(\mathcal{P}, C)$

where $\mathcal{P}$ is a compatible partition of T ,

$C \subseteq T$ is a clique, and $|\mathcal{P}| = |C|$

```

1: let  $t_0, \dots, t_{n-1}$  be an ordering of  $T$  such that  $s_{t_0} \leq \dots \leq s_{t_{n-1}}$ 
2:  $\mathcal{P} \leftarrow \emptyset$ 
3: let  $H$  be an empty priority queue
4: for  $r \leftarrow 0$  to  $n-1$  do
5:    $i \leftarrow t_r$ 
6:   if  $H \neq \emptyset$  and  $\min(H) \leq s_i$  then
7:      $(P, v) \leftarrow \text{remove}(H)$ 
8:      $\mathcal{P} \leftarrow (\mathcal{P} \setminus \{P\}) \cup \{P \cup \{i\}\}$ 
9:      $\text{insert}(H, (P \cup \{i\}, f_i))$ 
10:  else
11:     $k \leftarrow i$ 
12:     $P \leftarrow \{i\}$ 
13:     $\mathcal{P} \leftarrow \mathcal{P} \cup \{P\}$ 
14:     $\text{insert}(H, (P, f_i))$ 
15:  $C \leftarrow \emptyset$ 
16: for  $t \in T$  do
17:   if  $s_t \leq s_k < f_t$  then
18:      $C \leftarrow C \cup \{t\}$ 
19: return  $(\mathcal{P}, C)$ 

```

Exercise 2.11

For each $r \in [0 : n)$, let $T_r := \{t_0, \dots, t_r\}$. Show that the following invariant holds at the end of the iteration of the first for-loop:

- (i) $\mathcal{P}$ is a compatible partition of T_r , and
- (ii) there exists a clique $C \subseteq T_r$ such that $k \in C$ and $|C| = |\mathcal{P}|$.

2.4 Huffman coding

We will need some preliminary definitions.

We begin reviewing the notion of an alphabet and some related concepts. An *alphabet* is a finite and nonempty set of elements called *symbols*.

Let $n \in \mathbb{N}$, and let Σ be an alphabet. A *word* over Σ is a sequence $w : [0 : n) \rightarrow \Sigma$. It is *empty* if $n = 0$, and, in this case, we denote it by ϵ . The set of words over Σ is denoted by Σ^* , while the set of nonempty words is denoted by Σ^+ . Thus, $\Sigma^+ = \Sigma^* \setminus \{\epsilon\}$.

Let Σ be an alphabet. If $u : [0 : m) \rightarrow \Sigma$ and $v : [0 : n) \rightarrow \Sigma$ are words over Σ , the *concatenation* of u and v , denoted by uv , is the word $uv : [0 : m + n) \rightarrow \Sigma$ defined by

$$(uv)_i := \begin{cases} u_i, & \text{if } i \in [0 : m), \\ v_{i-m}, & \text{if } i \in [m : m + n). \end{cases}$$

In particular, $\epsilon u = u\epsilon = u$. We will often identify a symbol $a \in \Sigma$ with the word of length 1 whose only symbol is a . For a word $u \in \Sigma^+$, let $u^- \in \Sigma^*$ be the unique word for which there exists $a \in \Sigma$ such that $u = u^- a$. Thus, u^- is the word obtained from u by removing its last symbol.

Let u and v be words over Σ . We say that u is a *prefix* of v if there exists a word $w \in \Sigma^*$ such that $v = uw$. If, in addition, $u \neq v$, then u is called a *proper prefix* of v .

A pair (Σ, f) , where Σ is an alphabet and $f : \Sigma \rightarrow \mathbb{N}_+$, is called a *frequency table*. For each $a \in \Sigma$, the number $f(a)$ is the *frequency* of the symbol a .

A *binary encoding* of an alphabet Σ is a function $\sigma : \Sigma \rightarrow \mathbb{B}^+$, where $\mathbb{B} := \{0, 1\}$. Thus, for each symbol $a \in \Sigma$, the word $\sigma(a)$ is a sequence of zeros and ones; we call $\sigma(a)$ the *code* or *codeword* of a . We say that σ is *prefix-free* if, for all $a_1, a_2 \in \Sigma$ with $a_1 \neq a_2$, the word $\sigma(a_1)$ is not a prefix of $\sigma(a_2)$. Equivalently, σ is prefix-free if, for all $a_1, a_2 \in \Sigma$, whenever $\sigma(a_1)$ is a prefix of $\sigma(a_2)$, we have $a_1 = a_2$.

Let (Σ, f) be a frequency table, and let σ be a binary encoding of Σ . The *cost* of σ is the number

$$\text{cost}(\sigma) := \text{cost}_f(\sigma) := \sum_{a \in \Sigma} f(a) |\sigma(a)|.$$

We first investigate some properties of a minimum prefix-free binary encoding. Let (Σ, f) be a frequency table, and let σ be a prefix-free binary encoding of Σ . We say that two distinct symbols $a_1, a_2 \in \Sigma$ are *siblings* in σ if

$$\sigma(a_1)^- = \sigma(a_2)^-.$$

Minimum prefix-free binary encodings We can now state the problem we are interested in. Given a frequency table (Σ, f) , find a prefix-free binary encoding σ of Σ with minimum cost, that is, one for which $\text{cost}(\sigma) \leq \text{cost}(\mu)$ for every prefix-free binary encoding μ of Σ . Such an encoding is called a *minimum prefix-free binary encoding* of (Σ, f) . When the frequency table is clear from the context, we simply say that σ is a *minimum prefix-free binary encoding* of Σ .

The next lemma asserts that, in a minimum prefix-free binary encoding of an alphabet with at least two symbols, every symbol whose codeword has maximum length has a sibling.

Lemma 2.9. *Let (Σ, f) be a frequency table with $|\Sigma| \geq 2$. If σ is a minimum prefix-free binary encoding of Σ and $b \in \Sigma$ is such that $|\sigma(b)| = \max\{|\sigma(a)| \mid a \in \Sigma\}$, then b has a sibling in σ .*

Proof. Suppose that σ is a minimum prefix-free binary encoding of Σ and that $b \in \Sigma$ is such that $|\sigma(b)| = \max\{|\sigma(a)| \mid a \in \Sigma\}$. Let $w := \sigma(b)$ and suppose, for a contradiction, that b has no sibling in σ .

First we prove that $|w| \geq 2$. Assume that $|w| = 1$. Since $|\Sigma| \geq 2$, there exists $a \in \Sigma \setminus \{b\}$. By the choice of b and since $\sigma(a) \in \mathbb{B}^+$, we have $|\sigma(a)| = 1$. Thus, $\epsilon = \sigma(a)^- = \sigma(b)^-$, and so a and b are siblings in σ , a contradiction.

Therefore $|w^-| \geq 1$, and hence the following definition gives a binary encoding of Σ :

$$\sigma'(a) := \begin{cases} w^-, & \text{if } a = b, \\ \sigma(a), & \text{otherwise,} \end{cases}$$

for all $a \in \Sigma$. We claim that σ' is prefix-free.

Since σ is prefix-free, it is enough to check pairs involving b . Let $a \in \Sigma \setminus \{b\}$. Suppose first that $\sigma'(a)$ is a prefix of $\sigma'(b) = w^-$. Since $\sigma'(a) = \sigma(a)$ and w^- is a prefix of $w = \sigma(b)$, it follows that $\sigma(a)$ is a prefix of $\sigma(b)$, which contradicts the fact that σ is prefix-free.

Suppose now that $\sigma'(b) = w^-$ is a prefix of $\sigma'(a) = \sigma(a)$. If $\sigma(a) = w^-$, then $\sigma(a)$ is a prefix of $w = \sigma(b)$, again contradicting the fact that σ is prefix-free. Hence $|\sigma(a)| \geq |w^-| + 1 = |w|$. The choice of b implies that $|\sigma(a)| \leq |\sigma(b)|$, whence $|\sigma(a)| = |\sigma(b)|$. Since w^- is a prefix of $\sigma(a)$ and $\sigma(b) = w$, it follows that a and b are siblings in σ , contradicting our assumption. Therefore σ' is prefix-free.

Finally, the only codeword whose length changed is the codeword of b , and its length decreased by 1. Hence

$$\text{cost}(\sigma') = \sum_{a \in \Sigma \setminus \{b\}} f(a)|\sigma(a)| + f(b)(|\sigma(b)| - 1) = \text{cost}(\sigma) - f(b).$$

Since $f(b) > 0$, we have $\text{cost}(\sigma') < \text{cost}(\sigma)$, contradicting the minimality of σ . We therefore conclude that b has a sibling in σ . $\square$

Let σ be a binary encoding of an alphabet Σ . For distinct $a_1, a_2 \in \Sigma$, we denote by $\sigma[a_1 \leftrightarrow a_2]$ the binary encoding of Σ that is obtained from σ by exchanging the codewords of a_1 and a_2 , and leaving all other codewords unchanged. That is,

$$\sigma[a_1 \leftrightarrow a_2](a) := \begin{cases} \sigma(a), & \text{if } a \in \Sigma \setminus \{a_1, a_2\}, \\ \sigma(a_2), & \text{if } a = a_1, \\ \sigma(a_1), & \text{if } a = a_2 \end{cases}$$

for all $a \in \Sigma$. Notice that, if σ is prefix-free, then $\sigma[a_1 \leftrightarrow a_2]$ is also prefix-free.

Lemma 2.10. *Let (Σ, f) be a frequency table, and let σ be a binary encoding of Σ . Then, for all distinct $a_1, a_2 \in \Sigma$, if $f(a_1) \leq f(a_2)$ and $|\sigma(a_1)| \leq |\sigma(a_2)|$, then*

$$\text{cost}(\sigma[a_1 \leftrightarrow a_2]) \leq \text{cost}(\sigma).$$

Proof. The proof consists of a simple algebraic manipulation. Let σ' be the binary encoding $\sigma[a_1 \leftrightarrow a_2]$. Then

$$\begin{aligned} \text{cost}(\sigma') - \text{cost}(\sigma) &= \sum_{a \in \Sigma} f(a)|\sigma'(a)| - \sum_{a \in \Sigma} f(a)|\sigma(a)| \\ &= f(a_1)|\sigma(a_2)| + f(a_2)|\sigma(a_1)| - f(a_1)|\sigma(a_1)| - f(a_2)|\sigma(a_2)| \\ &= f(a_1)(|\sigma(a_2)| - |\sigma(a_1)|) + f(a_2)(|\sigma(a_1)| - |\sigma(a_2)|) \\ &= (f(a_1) - f(a_2))(|\sigma(a_2)| - |\sigma(a_1)|). \end{aligned}$$

By hypothesis, $f(a_1) - f(a_2) \leq 0$ and $|\sigma(a_2)| - |\sigma(a_1)| \geq 0$. Therefore $\text{cost}(\sigma') - \text{cost}(\sigma) \leq 0$, and hence

$$\text{cost}(\sigma[a_1 \leftrightarrow a_2]) = \text{cost}(\sigma') \leq \text{cost}(\sigma).$$

□

Lemma 2.11. *Let (Σ, f) be a frequency table with $|\Sigma| \geq 2$, and let $a_1, a_2 \in \Sigma$ be such that $a_1 \neq a_2$, $f(a_1) = \min\{f(a) \mid a \in \Sigma\}$, and $f(a_2) = \min\{f(a) \mid a \in \Sigma \setminus \{a_1\}\}$. Then there exists a minimum prefix-free binary encoding σ of Σ such that $|\sigma(a_1)| = \max\{|\sigma(a)| \mid a \in \Sigma\}$ and a_1 and a_2 are siblings in σ .*

Proof. Let σ be a minimum prefix-free binary encoding of Σ .⁷ Let $b_1 \in \Sigma$ be such that $|\sigma(b_1)| = \max\{|\sigma(a)| \mid a \in \Sigma\}$. By the choice of a_1 , we have $f(a_1) \leq f(b_1)$. Moreover, by the choice of b_1 , we have $|\sigma(a_1)| \leq |\sigma(b_1)|$. Thus, if $a_1 \neq b_1$, then, by Lemma 2.10, we have $\text{cost}(\sigma[a_1 \leftrightarrow b_1]) \leq \text{cost}(\sigma)$. Since σ is minimum, $\sigma[a_1 \leftrightarrow b_1]$ is also minimum. Replacing σ by this encoding if necessary, we may suppose that $a_1 = b_1$.

By Lemma 2.9, there exists $b_2 \in \Sigma \setminus \{a_1\}$ such that a_1 and b_2 are siblings in σ . In particular, $|\sigma(a_1)| = |\sigma(b_2)|$. By the choice of a_2 , we have $f(a_2) \leq f(b_2)$. Moreover, $|\sigma(a_2)| \leq |\sigma(b_2)|$. Once again, if $a_2 \neq b_2$, then, by Lemma 2.10, we have $\text{cost}(\sigma[a_2 \leftrightarrow b_2]) \leq \text{cost}(\sigma)$. Since σ is minimum, $\sigma[a_2 \leftrightarrow b_2]$ is also minimum. Replacing σ by this encoding if necessary, we may suppose that $a_2 = b_2$.

Therefore, there exists a minimum prefix-free binary encoding σ of Σ such that $|\sigma(a_1)| = \max\{|\sigma(a)| \mid a \in \Sigma\}$ and a_1 and a_2 are siblings in σ . □

⁷Why such an object exists? Consider the set $\{\text{cost}(\sigma) \mid \sigma \text{ is prefix-free binary encoding of } \Sigma\}$. This is a nonempty subset of $\mathbb{N}$, hence, by the well-ordering principle, it has a minimum element. Therefore there exists a prefix-free binary encoding σ of Σ such that $\text{cost}(\sigma) \leq \text{cost}(\mu)$ for all prefix-free binary encoding μ of Σ .

Contraction of a frequency table Let (Σ, f) be a frequency table, and let $a_1, a_2 \in \Sigma$ be distinct symbols. Let α be a symbol such that $\alpha \notin \Sigma$. The **contraction** of (Σ, f) at a_1 and a_2 into α is the frequency table

$$(\Sigma, f) / (a_1, a_2 \mapsto \alpha) := (\Sigma', f'),$$

where

$$\Sigma' := (\Sigma \setminus \{a_1, a_2\}) \cup \{\alpha\}$$

and $f' : \Sigma' \rightarrow \mathbb{N}_+$ is defined by

$$f'(a) := \begin{cases} f(a_1) + f(a_2), & \text{if } a = \alpha, \\ f(a), & \text{if } a \in \Sigma' \setminus \{\alpha\} \end{cases}$$

for all $a \in \Sigma'$.

Expansion of an encoding Let Σ be an alphabet, let $\alpha \in \Sigma$, and let a_1, a_2 be distinct symbols such that $a_1, a_2 \notin \Sigma$. Set

$$\Sigma' := (\Sigma \setminus \{\alpha\}) \cup \{a_1, a_2\}.$$

If $\sigma : \Sigma \rightarrow \mathbb{B}^+$ is a binary encoding of Σ , then the **expansion** of σ at α into a_1 and a_2 is the binary encoding

$$\sigma[\alpha \mapsto (a_1, a_2)] : \Sigma' \rightarrow \mathbb{B}^+$$

defined by

$$\sigma[\alpha \mapsto (a_1, a_2)](a) := \begin{cases} \sigma(a), & \text{if } a \in \Sigma \setminus \{\alpha\}, \\ \sigma(\alpha)0, & \text{if } a = a_1, \\ \sigma(\alpha)1, & \text{if } a = a_2 \end{cases}$$

for all $a \in \Sigma'$.

Lemma 2.12. *Let (Σ, f) be a frequency table, let $a_1, a_2 \in \Sigma$ be distinct symbols, and let $\alpha \notin \Sigma$. Let*

$$(\Sigma', f') := (\Sigma, f) / (a_1, a_2 \mapsto \alpha).$$

If σ' is a binary encoding of Σ' , and $\sigma := \sigma'[\alpha \mapsto (a_1, a_2)]$, then

$$\text{cost}_f(\sigma) = \text{cost}_{f'}(\sigma') + f(a_1) + f(a_2).$$

Proof. Let σ' be a binary encoding of Σ' , and let $\sigma := \sigma'[\alpha \mapsto (a_1, a_2)]$. By the definition of the expansion, we have $\sigma(a) = \sigma'(a)$ for every $a \in \Sigma \setminus \{a_1, a_2\}$, while

$\sigma(a_1) = \sigma'(\alpha)0$ and $\sigma(a_2) = \sigma'(\alpha)1$. Hence

$$\begin{aligned}
 \text{cost}_f(\sigma) &= \sum_{a \in \Sigma \setminus \{a_1, a_2\}} f(a)|\sigma(a)| + f(a_1)|\sigma(a_1)| + f(a_2)|\sigma(a_2)| \\
 &= \sum_{a \in \Sigma \setminus \{a_1, a_2\}} f(a)|\sigma'(a)| + f(a_1)(|\sigma'(\alpha)| + 1) + f(a_2)(|\sigma'(\alpha)| + 1) \\
 &= \sum_{a \in \Sigma' \setminus \{\alpha\}} f'(a)|\sigma'(a)| + (f(a_1) + f(a_2))|\sigma'(\alpha)| + f(a_1) + f(a_2) \\
 &= \sum_{a \in \Sigma' \setminus \{\alpha\}} f'(a)|\sigma'(a)| + f'(\alpha)|\sigma'(\alpha)| + f(a_1) + f(a_2) \\
 &= \text{cost}_{f'}(\sigma') + f(a_1) + f(a_2).
 \end{aligned}$$

□

Theorem 2.13. *Let (Σ, f) be a frequency table with $|\Sigma| \geq 3$, and let $a_1, a_2 \in \Sigma$ be such that $f(a_1) = \min\{f(a) \mid a \in \Sigma\}$, and $f(a_2) = \min\{f(a) \mid a \in \Sigma \setminus \{a_1\}\}$. Let α be a symbol such that $\alpha \notin \Sigma$, and let*

$$(\Sigma', f') := (\Sigma, f)/(a_1, a_2 \mapsto \alpha).$$

If σ' is a minimum prefix-free binary encoding of (Σ', f') , then $\sigma'[\alpha \mapsto (a_1, a_2)]$ is a minimum prefix-free binary encoding of (Σ, f) .

Proof. Suppose that σ' is a minimum prefix-free binary encoding of (Σ', f') . First notice that $\sigma'[\alpha \mapsto (a_1, a_2)]$ is prefix-free. Indeed, the words $\sigma'(\alpha)0$ and $\sigma'(\alpha)1$ are not prefixes of each other. Moreover, the words $\sigma'(b)$, with $b \in \Sigma' \setminus \{\alpha\}$, are pairwise incomparable by prefix, since they are codewords of σ' .

Now let $b \in \Sigma' \setminus \{\alpha\}$ and $r \in \{0, 1\}$. We show that $\sigma'(\alpha)r$ and $\sigma'(b)$ are not comparable by prefix. If $\sigma'(\alpha)r$ is a prefix of $\sigma'(b)$, then $\sigma'(\alpha)$ is also a prefix of $\sigma'(b)$, contradicting the fact that σ' is prefix-free. Now suppose that $\sigma'(b)$ is a prefix of $\sigma'(\alpha)r$. Since $\sigma'(b)$ is not a prefix of $\sigma'(\alpha)$, we must have $\sigma'(b) = \sigma'(\alpha)r$. But then $\sigma'(\alpha)$ is a prefix of $\sigma'(b)$, again contradicting the fact that σ' is prefix-free. Thus no codeword of the form $\sigma'(\alpha)r$ is comparable by prefix with any codeword $\sigma'(b)$, and therefore $\sigma'[\alpha \mapsto (a_1, a_2)]$ is prefix-free.

Set $\sigma := \sigma'[\alpha \mapsto (a_1, a_2)]$. We now prove that σ is minimum. By Lemma 2.11, there exists a minimum prefix-free binary encoding μ of Σ such that $|\mu(a_1)| = \max\{|\mu(a)| \mid a \in \Sigma\}$ and a_1 and a_2 are siblings in μ . Since a_1 and a_2 are siblings, there is a word $w \in \mathbb{B}^+$ such that, possibly after exchanging the names of a_1 and a_2 , $\mu(a_1) = w0$ and $\mu(a_2) = w1$.

Indeed, the common prefix w cannot be the empty word. If it were, then $\mu(a_1) = 0$ and $\mu(a_2) = 1$. Since $|\Sigma| \geq 3$, there exists $a_3 \in \Sigma \setminus \{a_1, a_2\}$. The word $\mu(a_3)$ is nonempty, and therefore it begins with either 0 or 1. Hence either $\mu(a_1)$ or $\mu(a_2)$ would be a prefix of $\mu(a_3)$, contradicting the fact that μ is prefix-free.

Define $\mu' : \Sigma' \rightarrow \mathbb{B}^+$ by

$$\mu'(a) := \begin{cases} \mu(a), & \text{if } a \in \Sigma \setminus \{a_1, a_2\}, \\ w, & \text{if } a = \alpha. \end{cases}$$

Then μ' is prefix-free. Indeed, if some $\mu(a)$, with $a \in \Sigma \setminus \{a_1, a_2\}$, is a prefix of w , then it is also a prefix of $w0 = \mu(a_1)$ and of $w1 = \mu(a_2)$, contradicting the fact that μ is prefix-free. Suppose, on the other hand, that w is a prefix of some $\mu(a)$, with $a \in \Sigma \setminus \{a_1, a_2\}$. If w is a proper prefix of $\mu(a)$, then $\mu(a)$ begins with either $w0$ or $w1$. Hence either $\mu(a_1) = w0$ or $\mu(a_2) = w1$ is a prefix of $\mu(a)$, a contradiction. Hence $w = \mu(a)$. But then $\mu(a)$ is a prefix of $\mu(a_1)$, again a contradiction. Thus μ' is prefix-free.

By Lemma 2.12, applied first to σ' and then to μ' , we have

$$\text{cost}_f(\sigma) = \text{cost}_{f'}(\sigma') + f(a_1) + f(a_2)$$

and

$$\text{cost}_f(\mu) = \text{cost}_{f'}(\mu') + f(a_1) + f(a_2).$$

Since σ' is minimum for (Σ', f') , we have $\text{cost}_{f'}(\sigma') \leq \text{cost}_{f'}(\mu')$. Therefore

$$\text{cost}_f(\sigma) = \text{cost}_{f'}(\sigma') + f(a_1) + f(a_2) \leq \text{cost}_{f'}(\mu') + f(a_1) + f(a_2) = \text{cost}_f(\mu).$$

But μ is minimum for (Σ, f) . Thus $\text{cost}_f(\sigma) = \text{cost}_f(\mu)$, and so σ is a minimum prefix-free binary encoding of Σ . $\square$

Exercise 2.12

Show that the previous theorem may fail when $|\Sigma| = 2$. More precisely, take $\Sigma = \{a_1, a_2\}$ and compare the encoding obtained by the construction of the theorem with a minimum prefix-free binary encoding of Σ .

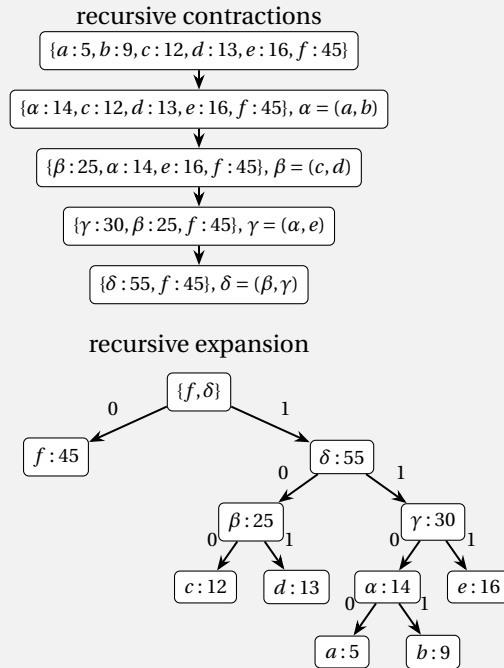
Algorithm 2.14 HUFFMAN(Σ, f)

Require: A frequency table (Σ, f) with $\Sigma \neq \emptyset$ **Ensure:** A minimum prefix-free binary encoding of Σ

```

1: if  $|\Sigma| = 1$  then
2:   let  $a$  be the unique element of  $\Sigma$ 
3:   return  $\{(a, 0)\}$ 
4: if  $|\Sigma| = 2$  then
5:   let  $a_1, a_2$  be the two elements of  $\Sigma$ 
6:   return  $\{(a_1, 0), (a_2, 1)\}$ 
7: choose  $a_1 \in \Sigma$  such that  $f(a_1) = \min\{f(a) \mid a \in \Sigma\}$ 
8: choose  $a_2 \in \Sigma \setminus \{a_1\}$  such that  $f(a_2) = \min\{f(a) \mid a \in \Sigma \setminus \{a_1\}\}$ 
9: choose a symbol  $\alpha$  such that  $\alpha \notin \Sigma$ 
10:  $(\Sigma', f') \leftarrow (\Sigma, f) / (a_1, a_2 \mapsto \alpha)$ 
11:  $\sigma' \leftarrow \text{HUFFMAN}(\Sigma', f')$ 
12: return  $\sigma'[\alpha \mapsto (a_1, a_2)]$ 

```



$$\sigma(f) = 0, \quad \sigma(c) = 100, \quad \sigma(d) = 101, \quad \sigma(a) = 1100, \quad \sigma(b) = 1101, \quad \sigma(e) = 111.$$

Figure 2.14: An illustration of the recursive structure of Huffman's algorithm. First, the two symbols of least frequency are repeatedly contracted until only two symbols remain. The base instance assigns the words 0 and 1 to these two symbols. The remaining contractions are then expanded in reverse order.

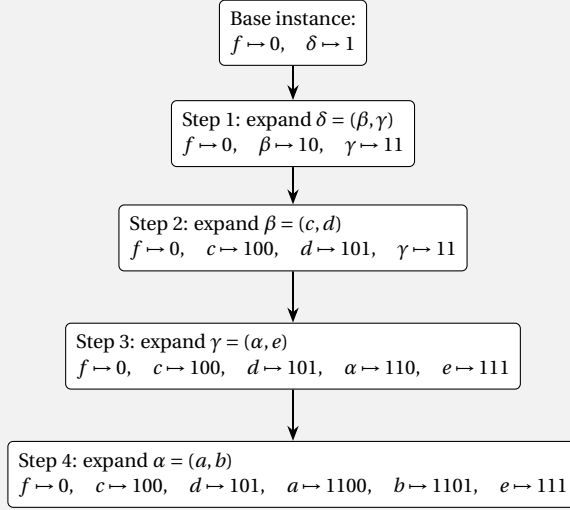


Figure 2.15: The expansion phase of the recursive version of Huffman's algorithm. Since the base case occurs when the alphabet has two symbols, the base instance assigns the words 0 and 1 to the two remaining symbols. The remaining contractions are then expanded in reverse order.

We will now present a more efficient version of Huffman's algorithm. This version uses a priority queue whose elements are pairs of the form (σ, p) , where σ is a partial encoding and p is the sum of the frequencies of the symbols encoded by σ .

We will need the following operations. Let Σ be an alphabet, let $\sigma : \Sigma \rightarrow \mathbb{B}^*$, and let $b \in \mathbb{B}$. We write $b\sigma$ to denote the function

$$b\sigma := \{(a, b\sigma(a)) \mid a \in \Sigma\}.$$

If $\sigma_0 : \Sigma_0 \rightarrow \mathbb{B}^*$ and $\sigma_1 : \Sigma_1 \rightarrow \mathbb{B}^*$ are functions with disjoint domains, we write

$$\sigma_0 \oplus \sigma_1 := 0\sigma_0 \cup 1\sigma_1.$$

Thus, $\sigma_0 \oplus \sigma_1$ is a function whose domain is $\Sigma_0 \cup \Sigma_1$.

Algorithm 2.15 HUFFMANFASTREC(H)

Require: A priority queue H whose elements are pairs (σ, p) **Ensure:** A prefix-free binary encoding

```

1: if  $|H| = 1$  then
2:    $(\sigma, p) \leftarrow \text{remove}(H)$ 
3:   return  $\sigma$ 
4:  $(\sigma_0, p) \leftarrow \text{remove}(H)$ 
5:  $(\sigma_1, q) \leftarrow \text{remove}(H)$ 
6:  $\text{insert}(H, (\sigma_0 \oplus \sigma_1, p + q))$ 
7: return HUFFMANFASTREC( $H$ )

```

Algorithm 2.16 HUFFMANFAST(Σ, f)

Require: A frequency table (Σ, f) with $\Sigma \neq \emptyset$ **Ensure:** A minimum prefix-free binary encoding of (Σ, f)

```

1: if  $|\Sigma| = 1$  then
2:   let  $a$  be the unique symbol of  $\Sigma$ 
3:   return  $\{(a, 0)\}$ 
4: let  $H$  be an empty priority queue
5: for each  $a \in \Sigma$  do
6:    $\text{insert}(H, \{(a, \epsilon)\}, f(a))$ 
7: return HUFFMANFASTREC( $H$ )

```

Let $n := |\Sigma|$. The priority queue operations contribute $O(n \lg n)$ time when the queue is implemented by a binary heap. Hence Algorithm 2.16 can be implemented to run in time $O(n \lg n)$, provided that the operation $\sigma_0 \oplus \sigma_1$ is implemented in constant time.

Now, we show an iterative version of Huffman's algorithm.

Algorithm 2.17 HUFFMANITERATIVE(Σ, f)**Require:** A frequency table (Σ, f) **Ensure:** A minimum prefix-free binary encoding of (Σ, f)

```

1: if  $|\Sigma| = 1$  then
2:   let  $a$  be the unique symbol in  $\Sigma$ 
3:   return  $\{(a, 0)\}$ 
4: let  $H$  be an empty priority queue
5: for each  $a \in \Sigma$  do
6:   insert( $H, (\{a, \epsilon\}, f(a))$ )
7: while  $|H| > 1$  do
8:    $(\sigma_0, p) \leftarrow \text{remove}(H)$ 
9:    $(\sigma_1, q) \leftarrow \text{remove}(H)$ 
10:  insert( $H, (\sigma_0 \oplus \sigma_1, p + q)$ )
11:  $(\sigma, p) \leftarrow \text{remove}(H)$ 
12: return  $\sigma$ 

```

2.4.1 The tree representation

In this section, we show how binary encodings can be represented by ordered binary trees. This representation is useful both for thinking about an implementation and for visualizing Huffman's algorithm.

We begin by defining the notion of a binary tree. A *binary tree* is a pair (T, r) , where $T := (V, A)$ is a digraph and $r \in V$ is a distinguished vertex, called the *root*, such that⁸

- (i) r has indegree 0,
- (ii) for every $v \in V$, there exists a unique path from r to v , and
- (iii) every vertex has outdegree at most 2.

A vertex with outdegree 0 is called a *leaf*. A vertex that is not a leaf is called an *internal vertex*.

An *ordered binary tree* is a triple (T, r, ℓ) , where $(T = (V, A), r)$ is a binary tree and $\ell : A \rightarrow \{0, 1\}$ is a labeling of the arcs such that, for every vertex $u \in V$, if $(u, v), (u, w) \in A$ and $v \neq w$, then

$$\{\ell(u, v), \ell(u, w)\} = \{0, 1\}.$$

For an arc $a \in A$, we say that $\ell(a)$ is the *label* of a . An arc labelled 0 is called a *left arc*, while an arc labelled 1 is called a *right arc*.

An ordered binary tree is *full* if every vertex has outdegree either 0 or 2.

⁸The *indegree* of a vertex v in a digraph $D := (V, A)$ is the number of arcs of D whose head is v ; similarly, the *outdegree* of v is the number of arcs of D whose tail is v .

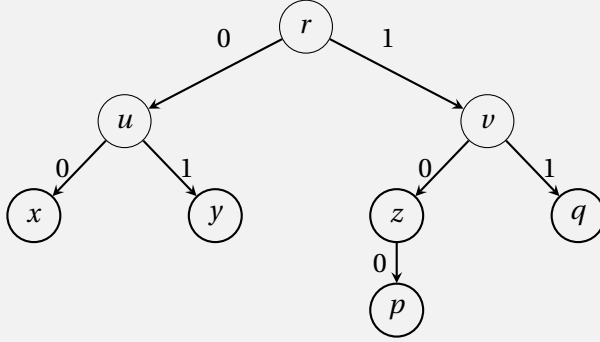


Figure 2.16: An ordered binary tree. The labels 0 and 1 distinguish the outgoing arcs of each internal vertex.

Let (T, r, ℓ) be an ordered binary tree, with $T = (V, A)$. For each $v \in V$, the **word** of v is the word obtained by reading the labels of the arcs in the unique path from r to v . More precisely, if $\langle u_0, u_1, \dots, u_k \rangle$ is the unique (r, v) -path in T , then the word of v is

$$\ell(u_0, u_1)\ell(u_1, u_2)\cdots\ell(u_{k-1}, u_k).$$

Notice that, if $v = r$, the path has no arcs and the corresponding concatenation is empty; thus the root has word ϵ , the empty word.

We now explain how binary encodings give rise to ordered binary trees. Let Σ be a finite alphabet, and let $\sigma : \Sigma \rightarrow \mathbb{B}^+$ be a binary encoding of Σ . Let P_σ be the set of all prefixes of the words $\sigma(a)$, with $a \in \Sigma$, including the empty word ϵ .

We define the **prefix ordered digraph** associated with σ as the triple $(T_\sigma, \epsilon, \ell_\sigma)$, where $T_\sigma := (P_\sigma, A_\sigma)$ and $\ell_\sigma : A_\sigma \rightarrow \mathbb{B}$ are defined as follows:

- (i) for $u, v \in P_\sigma$, we have $(u, v) \in A_\sigma$ if and only if there exists $x \in \mathbb{B}$ such that $v = ux$;
- (ii) for every $(u, v) \in A_\sigma$ and every $x \in \mathbb{B}$, we have $\ell_\sigma(u, v) = x$ if and only if $v = ux$.

Proposition 2.14. *Let Σ be a finite alphabet, and let $\sigma : \Sigma \rightarrow \mathbb{B}^+$ be a binary encoding of Σ . Then the prefix ordered digraph $(T_\sigma, \epsilon, \ell_\sigma)$ associated with σ is an ordered binary tree. Moreover, for every $a \in \Sigma$, the vertex $\sigma(a)$ of T_σ has word $\sigma(a)$.*

Proof. Let $T_\sigma := (P_\sigma, A_\sigma)$ be the prefix digraph associated with σ . We first prove that (T_σ, ϵ) is a binary tree.

Let $w \in P_\sigma$. First notice that $\langle v_0, v_1, \dots, v_k \rangle$, where $k := |w|$, $v_0 := \epsilon$, and $v_i := w_0 \cdots w_{i-1}$ for every $i \in [1 : k]$, is an (ϵ, w) -path in T_σ . Thus, for each $w \in P_\sigma$, there is at least one (ϵ, w) -path in T_σ .

Since $(u, v) \in A_\sigma$ implies $|v| = |u| + 1$, we have that, if $w \in P_\sigma \setminus \{\epsilon\}$, then every (ϵ, w) -path in T_σ has strictly positive length.

Next, we show that, for all $w \in P_\sigma$, if Q_1, Q_2 are (ϵ, w) -paths, then $Q_1 = Q_2$. The proof is by induction on $|w|$. Let $w \in P_\sigma$, and let Q_1, Q_2 be (ϵ, w) -paths in T_σ . If $w = \epsilon$, the result is immediate. Assume that $w \neq \epsilon$. Then there are unique $u \in \mathbb{B}^*$ and $x \in \mathbb{B}$ such that $w = ux$. Since $w \in P_\sigma$ and u is a prefix of w , we have $u \in P_\sigma$. Hence $(u, w) \in A_\sigma$ and $\ell_\sigma(u, w) = x$. Moreover, if $(u', w) \in A_\sigma$, then $u = u'$. Thus the penultimate vertex of both Q_1 and Q_2 is u . Removing the last vertex w from Q_1 and Q_2 , we obtain two (ϵ, u) -paths. By the induction hypothesis, these two paths are equal. Hence $Q_1 = Q_2$.

Now, ϵ has indegree 0. Indeed, there is no word $u \in P_\sigma$ and no bit $x \in \mathbb{B}$ such that $\epsilon = ux$. Moreover, every vertex has outdegree at most 2, since if $(u, v) \in A_\sigma$, then $v = ux$ for some $x \in \mathbb{B}$, and there are only two possible choices for x . Hence (T_σ, ϵ) is a binary tree. It remains to check that it is ordered. Let $(u, v), (u, w) \in A_\sigma$ be distinct arcs leaving the same vertex u . Then there exist $x, y \in \mathbb{B}$ such that $v = ux$ and $w = uy$. Since $v \neq w$, we have $x \neq y$, and hence $\{x, y\} = \{0, 1\}$. By the definition of ℓ_σ , this gives

$$\{\ell_\sigma(u, v), \ell_\sigma(u, w)\} = \{0, 1\}.$$

Therefore $(T_\sigma, \epsilon, \ell_\sigma)$ is an ordered binary tree.

Finally, if $a \in \Sigma$, then $\sigma(a) \in P_\sigma$. Since the unique path from ϵ to a word $w \in P_\sigma$ of length k is

$$\langle \epsilon, w_0, w_0 w_1, \dots, w_0 \cdots w_{k-1} = w \rangle,$$

the labels read along this path are precisely $w_0, w_1, \dots, w_{k-1}$. Hence the word of the vertex $\sigma(a)$ is exactly $\sigma(a)$. $\square$

Thus every binary encoding can be represented by an ordered binary tree. The vertices corresponding to codewords need not be leaves. This happens because one codeword may be a prefix of another codeword.

For example, consider the binary encoding given by

$$\sigma(a) = 0, \quad \sigma(b) = 01, \quad \sigma(c) = 100, \quad \sigma(d) = 101, \quad \sigma(e) = 11.$$

The associated prefix digraph is represented in Figure 2.17. The highlighted vertices correspond to codewords. Notice that this code is not prefix-free, since $\sigma(a) = 0$ is a prefix of $\sigma(b) = 01$.

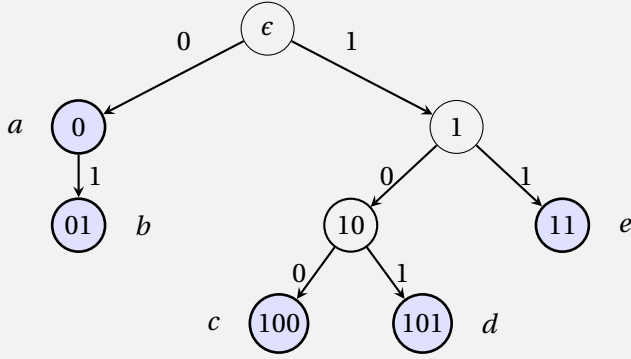


Figure 2.17: The prefix digraph associated with the binary code $\sigma(a) = 0$, $\sigma(b) = 01$, $\sigma(c) = 100$, $\sigma(d) = 101$, and $\sigma(e) = 11$. The highlighted vertices correspond to codewords.

The prefix-free condition has a simple interpretation in this representation.

Proposition 2.15. *Let Σ be a finite alphabet, and let $\sigma : \Sigma \rightarrow \mathbb{B}^+$ be an injective binary encoding of Σ . Then σ is prefix-free if and only if every vertex of the prefix digraph corresponding to a codeword $\sigma(a)$, with $a \in \Sigma$, is a leaf.*

Proof. Let $(T_\sigma, \epsilon, \ell_\sigma)$ be the prefix digraph associated with σ . Suppose first that σ is prefix-free, and let $a \in \Sigma$. If the vertex $\sigma(a)$ were not a leaf in T_σ , then it would have a descendant $w \neq \sigma(a)$. Since $w \in P_\sigma$, there exists $b \in \Sigma$ such that w is a prefix of $\sigma(b)$. Hence $\sigma(a)$ is a proper prefix of $\sigma(b)$, contradicting the fact that σ is prefix-free. Therefore every vertex corresponding to a codeword is a leaf.

Conversely, suppose that every vertex corresponding to a codeword is a leaf. Let $a_1, a_2 \in \Sigma$ be such that $a_1 \neq a_2$. The injectivity of σ implies that $\sigma(a_1) \neq \sigma(a_2)$. If $\sigma(a_1)$ is a prefix of $\sigma(a_2)$, then $\sigma(a_1)$ is a proper prefix of $\sigma(a_2)$, and therefore the vertex $\sigma(a_1)$ has a descendant, namely $\sigma(a_2)$. This contradicts the fact that $\sigma(a_1)$ is a leaf. Therefore, for distinct $a_1, a_2 \in \Sigma$, the word $\sigma(a_1)$ is not a prefix of $\sigma(a_2)$. Hence σ is prefix-free. $\square$

We will now show a version of Huffman's algorithm that returns an ordered binary tree instead of a prefix-free binary encoding. For this, we need the following definition.

Let $B_0 := ((V_0, A_0), r_0, \ell_0)$ and $B_1 := ((V_1, A_1), r_1, \ell_1)$ be ordered binary trees with disjoint vertex sets. We write $B_0 \oplus B_1$ to denote any ordered binary tree $((V, A), r, \ell)$ satisfying:

- (i) $r \notin V_0 \cup V_1$;
- (ii) $V := \{r\} \cup V_0 \cup V_1$;
- (iii) $A := A_0 \cup A_1 \cup \{(r, r_0), (r, r_1)\}$;

(iv) $\ell := \ell_0 \cup \ell_1 \cup \{(r, r_0), 0\}, \{(r, r_1), 1\}$.

Thus, B_0 becomes the left subtree of the new root, and B_1 becomes the right subtree. Different choices of r produce isomorphic ordered binary trees.

Algorithm 2.18 HUFFMANTREE(Σ, f)

Require: A frequency table (Σ, f) with $\Sigma \neq \emptyset$

Ensure: An ordered binary tree whose leaves are the symbols of Σ

```

1: if  $|\Sigma| = 1$  then
2:   let  $a$  be the unique symbol of  $\Sigma$ 
3:   let  $r$  be an object such that  $r \neq a$ 
4:   return  $((\{r, a\}, \{(r, a)\}), r, \{(r, a), 0\})$ 
5:  $H \leftarrow \emptyset$ 
6: for each  $a \in \Sigma$  do
7:    $B_a \leftarrow ((\{a\}, \emptyset), a, \emptyset)$ 
8:   insert( $H, (B_a, f(a))$ )
9: while  $|H| > 1$  do
10:   $(B_0, p) \leftarrow \text{remove}(H)$ 
11:   $(B_1, q) \leftarrow \text{remove}(H)$ 
12:  insert( $H, (B_0 \oplus B_1, p + q)$ )
13:  $(B, p) \leftarrow \text{remove}(H)$ 
14: return  $B$ 

```

Using a binary heap for the priority queue and a pointer-based representation of ordered binary trees, Algorithm 2.18 can be implemented to run in time $O(n \lg n)$, where $n := |\Sigma|$.

2.5 The fractional knapsack problem

In this section, we consider a variant of the knapsack problem in which items may be taken fractionally. More precisely, each item has a weight and a value, and we are allowed to take any fraction of each item.

A *fractional knapsack instance* is a quadruple (I, w, v, C) , where I is a finite set whose elements are called *items*, $C \in \mathbb{R}_+$, and $w, v : I \rightsquigarrow \mathbb{R}_{++}$, that is, for each $i \in I$, we have $w(i) \in \mathbb{R}_{++}$ and $v(i) \in \mathbb{R}_{++}$. For each item $i \in I$, the number $w_i := w(i)$ is called the *weight* of i , while $v_i := v(i)$ is called the *value* of i . The number C is called the *capacity*.

For each item $i \in I$, the number $e_i := v_i / w_i$ is called the *efficiency* of i .

A *fractional knapsack* for (I, w, v, C) is a function $x : I \rightarrow [0, 1]$. For each $i \in I$, the number $x_i := x(i)$ represents the fraction of item i placed in the knapsack.

The *weight* of x is

$$w(x) := \sum_{i \in I} w_i x_i,$$

and the *value* of x is

$$v(x) := \sum_{i \in I} v_i x_i.$$

We say that x is *feasible* if $w(x) \leq C$.

The *fractional knapsack problem* consists of finding a feasible fractional knapsack $x : I \rightarrow [0, 1]$ of maximum value.

Lemma 2.16. *Let (I, w, v, C) be a fractional knapsack instance, and let $x : I \rightarrow [0, 1]$ be a feasible fractional knapsack. Suppose that $i, j \in I$ are distinct items such that $x_i > 0$, $x_j < 1$, and $e_j \geq e_i$. Then there exists a feasible fractional knapsack x' such that*

- (i) $x'_j = 1$ or $x'_i = 0$,
- (ii) $x'_k = x_k$ for all $k \in I \setminus \{i, j\}$, and
- (iii) $v(x') \geq v(x)$.

Proof. Notice that item i contributes weight $w_i x_i$ to x . We want to decrease x_i by a positive value δ , with $0 < \delta \leq x_i$. This removes weight $w_i \delta$ from the knapsack. We then add the same amount of weight using item j . Thus, we need to find λ such that $w_j \lambda = w_i \delta$, whence $\lambda = (w_i / w_j) \delta$.

Now we must have $x_j + \lambda \leq 1$, and therefore

$$\delta \leq (1 - x_j) \frac{w_j}{w_i}.$$

Hence, let

$$\delta := \min \left\{ x_i, \frac{w_j}{w_i} (1 - x_j) \right\}.$$

Since $x_i > 0$ and $x_j < 1$, the number δ is positive.

Define $x' : I \rightarrow [0, 1]$ by

$$x'_k := \begin{cases} x_i - \delta, & \text{if } k = i, \\ x_j + \frac{w_i}{w_j} \delta, & \text{if } k = j, \\ x_k, & \text{otherwise.} \end{cases}$$

By the choice of δ , we have $x'_i \geq 0$ and $x'_j \leq 1$. Moreover, if $\delta = x_i$, then $x'_i = 0$, while if

$$\delta = \frac{w_j}{w_i} (1 - x_j),$$

then $x'_j = 1$. Since δ is the minimum of these two numbers, at least one of these equalities holds. Clearly, $x'_k = x_k$ for every $k \in I \setminus \{i, j\}$. Hence x' is a fractional knapsack, and properties (i) and (ii) hold.

Moreover,

$$w_i x'_i + w_j x'_j = w_i (x_i - \delta) + w_j \left(x_j + \frac{w_i}{w_j} \delta \right) = w_i x_i + w_j x_j.$$

Thus $w(x') = w(x)$. Since x is feasible, it follows that x' is also feasible.

It remains to compare the values of x and x' . We have

$$v(x') - v(x) = -v_i \delta + v_j \frac{w_i}{w_j} \delta = w_i \delta \left(\frac{v_j}{w_j} - \frac{v_i}{w_i} \right) = w_i \delta (e_j - e_i).$$

Since $e_j \geq e_i$, we obtain $v(x') \geq v(x)$. This proves (iii). $\square$

Lemma 2.17. *Let (I, w, v, C) be a fractional knapsack instance with $I \neq \emptyset$, and let $j \in I$ be an item of maximum efficiency. Let*

$$\alpha := \min \left\{ 1, \frac{C}{w_j} \right\}.$$

For every feasible fractional knapsack $x : I \rightarrow [0, 1]$, there exists a feasible fractional knapsack $z : I \rightarrow [0, 1]$ such that $z_j = \alpha$ and $v(z) \geq v(x)$.

Proof. First notice that, if $y : I \rightarrow [0, 1]$ is feasible, then $y_j \leq \alpha$. Indeed, $y_j \leq 1$, and

$$w_j y_j \leq w(y) \leq C,$$

so $y_j \leq C/w_j$. Hence $y_j \leq \min\{1, C/w_j\} = \alpha$.

For a feasible fractional knapsack $x : I \rightarrow [0, 1]$, let

$$\eta(x) := |\{k \in I \setminus \{j\} \mid x_k > 0\}|.$$

We prove the lemma by induction on $\eta(x)$.

Let $x : I \rightarrow [0, 1]$ be a feasible fractional knapsack. If $x_j = \alpha$, there is nothing to prove. Thus, suppose that $x_j < \alpha$.

If $\eta(x) = 0$, then $x_k = 0$ for every $k \in I \setminus \{j\}$. Define $z : I \rightarrow [0, 1]$ by $z_j := \alpha$ and $z_k := 0$ for every $k \in I \setminus \{j\}$. Since $w_j \alpha \leq C$, the fractional knapsack z is feasible. Moreover, since $x_j < \alpha$ and $v_j > 0$, we have $v(z) \geq v(x)$.

Assume now that $\eta(x) > 0$. Then there exists $i \in I \setminus \{j\}$ such that $x_i > 0$. Since j has maximum efficiency, we have $e_j \geq e_i$. Also, since $x_j < \alpha \leq 1$, we have $x_j < 1$. By Lemma 2.16, there exists a feasible fractional knapsack x' such that

- (i) $x'_j = 1$ or $x'_i = 0$,
- (ii) $x'_k = x_k$ for all $k \in I \setminus \{i, j\}$, and

(iii) $v(x') \geq v(x)$.

If $x'_j = 1$, then, since x' is feasible, the observation at the beginning of the proof gives $1 = x'_j \leq \alpha$. As $\alpha \leq 1$, we get $\alpha = 1$, and therefore $x'_j = \alpha$. Taking $z := x'$, we are done.

Thus we may suppose that $x'_i = 0$. By (ii), all coordinates outside $\{i, j\}$ remain unchanged, and therefore

$$\eta(x') < \eta(x).$$

By the induction hypothesis, there exists a feasible fractional knapsack $z : I \rightarrow [0, 1]$ such that $z_j = \alpha$ and $v(z) \geq v(x')$. Since $v(x') \geq v(x)$, it follows that $v(z) \geq v(x)$, as required. $\square$

Theorem 2.18. *Let (I, w, v, C) be a fractional knapsack instance with $I \neq \emptyset$, and let $j \in I$ be an item of maximum efficiency. Let*

$$\alpha := \min \left\{ 1, \frac{C}{w_j} \right\}$$

and $I' := I \setminus \{j\}$. If x' is an optimal fractional knapsack for $(I', w, v, C - w_j\alpha)$, then the function $x := x' \cup \{(j, \alpha)\}$ is an optimal fractional knapsack for (I, w, v, C) .

Proof. First notice that x is feasible for (I, w, v, C) . Indeed, since x' is feasible for $(I', w, v, C - w_j\alpha)$, we have

$$\sum_{i \in I'} w_i x'_i \leq C - w_j \alpha.$$

Therefore,

$$w(x) = w_j \alpha + \sum_{i \in I'} w_i x'_i \leq C.$$

Now let $y : I \rightarrow [0, 1]$ be any feasible fractional knapsack for (I, w, v, C) . By Lemma 2.17, there exists a feasible fractional knapsack $z : I \rightarrow [0, 1]$ such that $z_j = \alpha$ and $v(z) \geq v(y)$.

Let $z' : I' \rightarrow [0, 1]$ be defined by $z'_i := z_i$ for every $i \in I'$. Since $z_j = \alpha$, we have

$$\sum_{i \in I'} w_i z'_i = \sum_{i \in I'} w_i z_i = w(z) - w_j \alpha \leq C - w_j \alpha.$$

Thus z' is feasible for $(I', w, v, C - w_j\alpha)$. Since x' is optimal for $(I', w, v, C - w_j\alpha)$, we have

$$v(x') \geq v(z').$$

Therefore,

$$v(x) = v_j \alpha + v(x') \geq v_j \alpha + v(z') = v(z) \geq v(y).$$

Since y was an arbitrary feasible fractional knapsack for (I, w, v, C) , it follows that x is optimal for (I, w, v, C) . $\square$

Algorithm 2.19 FRACTIONAL-KNAPSACK(I, w, v, C)

Require: A fractional knapsack instance (I, w, v, C) **Ensure:** An optimal fractional knapsack for (I, w, v, C)

```

1: if  $I = \emptyset$  then
2:   return the empty function
3: choose  $j \in I$  such that  $e_j \geq e_i$  for every  $i \in I$ 
4:  $\alpha \leftarrow \min \left\{ 1, \frac{C}{w_j} \right\}$ 
5:  $I' \leftarrow I \setminus \{j\}$ 
6:  $x' \leftarrow \text{FRACTIONAL-KNAPSACK}(I', w, v, C - w_j \alpha)$ 
7: return  $x' \cup \{(j, \alpha)\}$ 

```

An iterative version of the algorithm now follows. It is clear that it is possible to implement the algorithm to run in time $O(n \lg n)$, where $n := |I|$.

Algorithm 2.20 FRACTIONAL-KNAPSACK-ITERATIVE(I, w, v, C)

Require: A fractional knapsack instance (I, w, v, C) **Ensure:** An optimal fractional knapsack for (I, w, v, C)

```

1: order the items as  $i_1, i_2, \dots, i_n$  so that  $e_{i_1} \geq e_{i_2} \geq \dots \geq e_{i_n}$ 
2:  $R \leftarrow C$ 
3: for each  $i \in I$  do
4:    $x_i \leftarrow 0$ 
5: for  $r = 1, 2, \dots, n$  do
6:   if  $R = 0$  then
7:     return  $x$ 
8:    $i \leftarrow i_r$ 
9:    $x_i \leftarrow \min \left\{ 1, \frac{R}{w_i} \right\}$ 
10:   $R \leftarrow R - w_i x_i$ 
11: return  $x$ 

```

2.6 More exercises

In the exercises below, the goal is not merely to find an algorithm. A complete solution should make explicit the greedy choice, the subproblem left after this choice, and the proof that the greedy choice is safe. More precisely, it should prove that the greedy choice can be combined with an optimal solution to the subproblem to form an optimal solution to the original problem. The algorithm itself should be clearly described, and its time complexity should be analyzed.

Exercise 2.13

Let $A := \langle a_{ij} \mid i \in [0 : m), j \in [0 : n) \rangle$ be a matrix of real numbers. A *row cover* of A is a set

$$C \subseteq [0 : m) \times [0 : n)$$

such that, for each $i \in [0 : m)$, there exists exactly one $j \in [0 : n)$ with $(i, j) \in C$. The weight of C is

$$w(C) := \sum_{(i,j) \in C} a_{ij}.$$

Find a row cover of maximum weight.

Exercise 2.14

Let $A := \langle a_{ij} \mid i \in [0 : m), j \in [0 : n) \rangle$ be a matrix of real numbers. A *partial row cover* of A is a set

$$C \subseteq [0 : m) \times [0 : n)$$

such that, for each $i \in [0 : m)$, there exists at most one $j \in [0 : n)$ with $(i, j) \in C$. The weight of C is

$$w(C) := \sum_{(i,j) \in C} a_{ij}.$$

Let $k \in [0 : m]$. Find a partial row cover of maximum weight among all partial row covers with exactly k elements.

Exercise 2.15

Let I be a finite set, and let $s, t : I \rightarrow \mathbb{R}$ be such that $s_i \leq t_i$ for every $i \in I$, where $s_i := s(i)$ and $t_i := t(i)$. Let $a, b \in \mathbb{R}$ be such that $a \leq b$. A subset $C \subseteq I$ is a *cover* of the closed interval $[a, b]$ if

$$[a, b] \subseteq \bigcup_{i \in C} [s_i, t_i].$$

Assume that $[a, b]$ can be covered by the intervals indexed by I . Find a cover of $[a, b]$ with minimum cardinality.

Exercise 2.16

Let B be a finite set of boxes and let O be a finite set of objects. Let $c : B \rightarrow \mathbb{R}_+$ be a capacity function, which assigns a capacity $c_b := c(b)$ to each box $b \in B$, and let $s : O \rightarrow \mathbb{R}_+$ be a size function, which assigns a size $s_o := s(o)$ to each object $o \in O$.

A *feasible matching* is a set $M \subseteq B \times O$ such that

- (i) for each box $b \in B$, there exists at most one object $o \in O$ such that $(b, o) \in M$;
- (ii) for each object $o \in O$, there exists at most one box $b \in B$ such that $(b, o) \in M$;
- (iii) for every $(b, o) \in M$, we have $s_o \leq c_b$.

Find a feasible matching of maximum cardinality.

Exercise 2.17

Consider a sequence $\langle x_0, x_1, \dots, x_{n-1} \rangle$ of real numbers, where n is even. Let $\mathcal{P}$ be a partition of the index set $[0 : n)$ into parts of size 2. The *value* of $\mathcal{P}$ is the number

$$\max\{x_i + x_j \mid \{i, j\} \in \mathcal{P}\}.$$

Design an algorithm that finds a partition of minimum value. Prove the correctness of your algorithm and determine its time complexity.

Exercise 2.18

Let J be a finite set of jobs, and let $n := |J|$. Also, let $t, w : J \rightarrow \mathbb{R}_{++}$ be functions. For each $j \in J$, we call $t(j)$ the *processing time* of j , and $w(j)$ the *weight* of j . A *schedule* of J is an ordering of the elements of J , that is, a bijection $\sigma : [0 : n) \rightarrow J$. For a schedule σ and a job $j \in J$, the *completion time* of j with respect to σ is

$$c_\sigma(j) := \sum_{i \in [0 : \sigma^{-1}(j)]} t(\sigma(i)).$$

The *value* of σ is

$$\sum_{j \in J} w(j) c_\sigma(j).$$

Design an efficient algorithm that returns a schedule of minimum value. Prove that your algorithm is correct and determine its time complexity.

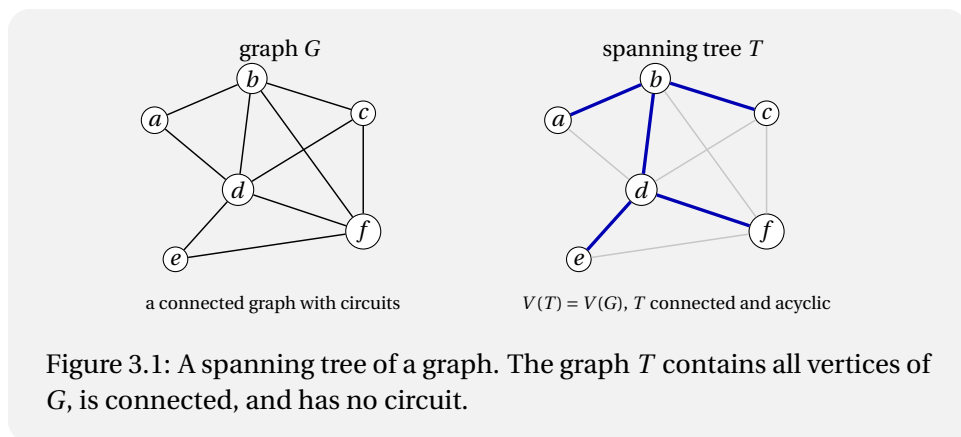
Suggestion: Pay attention to the quotient $w(j) / t(j)$.

Chapter 3

Minimum spanning trees

3.1 Spanning trees

Let G be a graph. We say that a spanning subgraph T of G is a *spanning tree* of G if T is a tree.



Recall that a *cut* in a graph G is a subset $K \subseteq E(G)$ such that $K = \delta(X)$ for some subset $X \subseteq V(G)$. The next algorithm describes a nondeterministic procedure that receives a graph G and builds a subset F of $E(G)$. At each iteration, the algorithm nondeterministically chooses a nonempty cut K disjoint from F , and then chooses an edge $f \in K$ to be added to F . We will show that this algorithm halts after a finite number of steps and that, whenever the input graph is connected, the set of edges returned by the algorithm is the edge set of a spanning tree of G .

Algorithm 3.1 CUTGREEDY(G)

```

1:  $F \leftarrow \emptyset$ 
2: while there exists a nonempty cut  $K$  disjoint from  $F$  do
3:   choose  $f \in K$ 
4:    $F \leftarrow F \cup \{f\}$ 
5: return  $F$ 

```

Proposition 3.1. *Let G be a graph. Then $(V(G), F)$ is acyclic at the beginning of every iteration of Algorithm CUTGREEDY(G).*

Proof. At the beginning of the algorithm, $F = \emptyset$, and therefore $(V(G), F)$ is acyclic.

Suppose now that $(V(G), F)$ is acyclic at the beginning of some iteration. Let K be a nonempty cut of G such that $K \cap F = \emptyset$, and let $xy := f \in K$ be the edge selected by the algorithm. We prove that $(V(G), F \cup \{f\})$ is acyclic.

Suppose, for a contradiction, that $(V(G), F \cup \{f\})$ contains a circuit C . Since $(V(G), F)$ is acyclic, the circuit C must contain the edge f . Hence $(V(G), F)$ contains an (x, y) -path $\langle u_0, \dots, u_k \rangle$. Let $X \subseteq V(G)$ be such that $K = \delta_G(X)$. Renaming x and y , if necessary, we may suppose that $x \in X$ and $y \in V(G) \setminus X$. Since $u_0 = x \in X$ and $u_k = y \in V(G) \setminus X$, there exists $i \in [0 : k]$ such that $u_i \in X$ and $u_{i+1} \in V(G) \setminus X$. Thus $u_i u_{i+1} \in \delta_G(X) = K$. On the other hand, since $\langle u_0, \dots, u_k \rangle$ is a path in $(V(G), F)$, we have $u_i u_{i+1} \in F$. Therefore $u_i u_{i+1} \in K \cap F$, contradicting the fact that $K \cap F = \emptyset$.

Therefore $(V(G), F \cup \{f\})$ is acyclic. Thus the invariant is preserved at every iteration. $\square$

Lemma 3.2. *Let G be a connected graph. If $F \subseteq E(G)$ is such that $(V(G), F)$ is acyclic and $|F| < |V(G)| - 1$, then there exists a nonempty cut K of G such that $K \cap F = \emptyset$.*

Proof. Suppose that $F \subseteq E(G)$ is such that $H := (V(G), F)$ is acyclic and $|F| < |V(G)| - 1$. If H were a spanning tree of G , then, by Theorem 1.7, we would have

$$|F| = |E(H)| = |V(H)| - 1 = |V(G)| - 1,$$

contrary to the hypothesis. Hence H is not a spanning tree of G .

Since H is acyclic, it follows that H is not connected. Therefore there exist vertices $x, y \in V(G)$ such that H contains no (x, y) -path. Let $X := [x]_H$, that is,

$$X = \{z \in V(H) \mid \text{there exists an } (x, z)\text{-path in } H\}.$$

Then $x \in X$ and $y \notin X$, whence $\emptyset \neq X \subset V(G)$. Moreover, $\delta_H(X) = \emptyset$, by the definition of X .

Since G is connected and $\emptyset \neq X \subset V(G)$, we have, by Theorem 1.10, that $\delta_G(X) \neq \emptyset$. Set $K := \delta_G(X)$. Then K is a nonempty cut of G . Moreover, since

$\delta_H(X) = \emptyset$ and $E(H) = F$, no edge of F has one endpoint in X and the other in $V(G) \setminus X$. Hence $K \cap F = \emptyset$. $\square$

Corollary 3.3. *For each connected graph G , Algorithm 3.1 executes $|V(G)| - 1$ iterations and returns a set $F \subseteq E(G)$ such that $(V(G), F)$ is a spanning tree of G .*

Proof. By the invariant proved above, throughout the execution of the algorithm, the graph $(V(G), F)$ is acyclic. Moreover, $|F|$ increases by 1 at each iteration.

We first show that the algorithm cannot halt while $|F| < |V(G)| - 1$. Indeed, by Lemma 3.2, if $|F| < |V(G)| - 1$, then there exists a nonempty cut K of G such that $K \cap F = \emptyset$. Thus the condition of the while loop is still satisfied.

Since $|F|$ increases by 1 at each iteration, the algorithm eventually reaches an iteration count for which $|F| = |V(G)| - 1$. At this point, $H := (V(G), F)$ is acyclic and $|F| = |V(G)| - 1$. Hence, by Theorem 1.8, the graph H is a tree. Since $V(H) = V(G)$, it is a spanning tree of G .

It remains to show that the algorithm halts at this point. Let K be a nonempty cut of G . Choose $X \subseteq V(G)$ such that $K = \delta_G(X)$. Since $K \neq \emptyset$, the set X is a nonempty proper subset of $V(G)$. Because H is connected, by Theorem 1.10,

$$\delta_H(X) = \delta_H(X) \cap F \neq \emptyset.$$

But H is a subgraph of G . Therefore

$$\delta_H(X) \cap F \subseteq \delta_G(X) \cap F = K \cap F.$$

Thus $K \cap F \neq \emptyset$. Hence there is no nonempty cut of G disjoint from F , and therefore the algorithm halts.

Thus the algorithm executes $|V(G)| - 1$ iterations and returns a set $F \subseteq E(G)$ such that $(V(G), F)$ is a spanning tree of G . $\square$

Lemma 3.4. *Let T be a tree, and write $V := V(T)$. If $x, y \in V$ are distinct vertices, $e := xy \notin E(T)$, and $f \in E(T(x, y))$, then $T - f + e$ is also a tree.*

Proof. Suppose that T is a tree, that x and y are distinct vertices of T , that $e := xy \notin E(T)$, and that $f \in E(T(x, y))$. Write $T(x, y) = \langle z_0, \dots, z_\ell \rangle$, where $z_0 = x$ and $z_\ell = y$. Let $T' := T - f + e$.

Since f is an edge of $T(x, y)$, there exists $j \in [0 : \ell]$ such that $f = z_j z_{j+1}$. Now consider the sequence $\langle z_{j+1}, z_{j+2}, \dots, z_\ell, z_0, z_1, \dots, z_j \rangle$. This is a (z_{j+1}, z_j) -path in T' . Indeed, all its edges, except for $z_\ell z_0$, are edges of $T(x, y)$ distinct from f , and $z_\ell z_0 = xy = e$. Thus T' contains a path between the two endpoints of f . In particular, $z_j \sim_{T'} z_{j+1}$.

Notice that $|E(T')| = |E(T)| = |V(T)| - 1 = |V(T')| - 1$. To complete the proof that T' is a tree, by Theorem 1.9, it remains to show that T' is connected.

Let $u, v \in V(T')$. Since T is connected, there exists an (u, v) -path

$$P := \langle u_0, \dots, u_k \rangle$$

in T , where $u_0 = u$ and $u_k = v$. If $f \notin E(P)$, then P is also an (u, v) -path in T' . Assume then that $f \in E(P)$. There exists $i \in [0 : k)$ such that $f = u_i u_{i+1}$. The initial segment $\langle u_0, \dots, u_i \rangle$ does not use f , and hence is a path in T' . Similarly, the final segment $\langle u_{i+1}, \dots, u_k \rangle$ is a path in T' . Moreover, by the first part of the proof, the two endpoints of f are connected by a path in T' . Therefore $u_0 \sim_{T'} u_i$, $u_i \sim_{T'} u_{i+1}$, and $u_{i+1} \sim_{T'} u_k$. By the transitivity of reachability, it follows that $u \sim_{T'} v$.

Thus T' is connected and $|E(T')| = |V(T')| - 1$, and so, by Theorem 1.9, T' is a tree. $\square$

Now we turn to the minimum spanning tree problem.

3.2 The minimum spanning tree problem

Let G be a graph, and let $c : E(G) \rightsquigarrow \mathbb{R}$ be a function. The pair (G, c) is called a *weighted graph*, and $c(e)$ is called the *weight* of the edge e .

For a subset $F \subseteq E(G)$, we define

$$\tilde{c}(F) := \sum_{f \in F} c(f).$$

If H is a subgraph of G , we also write $\tilde{c}(H) := \tilde{c}(E(H))$.

Given a weighted graph (G, c) , with G connected, the *minimum spanning tree problem* consists of finding a spanning tree T of G such that

$$\tilde{c}(T) \leq \tilde{c}(J)$$

for every spanning tree J of G . Such a spanning tree T is called a *minimum spanning tree* of (G, c) .

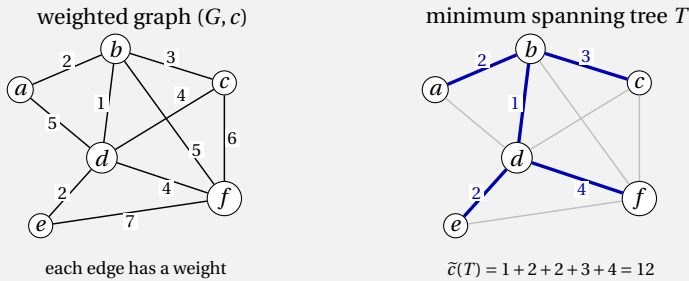


Figure 3.2: The minimum spanning tree problem. Given a connected weighted graph (G, c) , the goal is to find a spanning tree T of minimum total weight. The highlighted edges on the right form one such minimum spanning tree.

The next theorem is the key step in the proof that a greedy choice is enough to compute a minimum spanning tree of a weighted connected graph. Recall Algorithm 3.1, which nondeterministically selects two objects at each iteration: a nonempty cut and an edge in it. We keep the nondeterministic choice of the cut. However, once the cut has been chosen, we choose an edge in the cut with minimum weight. This gives the following version of that algorithm.

Algorithm 3.2 MSTGREEDY(G, c)

```

1:  $F \leftarrow \emptyset$ 
2: while there exists a nonempty cut  $K$  disjoint from  $F$  do
3:   choose  $e \in K$  such that  $c(e) = \min\{c(f) \mid f \in K\}$ 
4:    $F \leftarrow F \cup \{e\}$ 
5: return  $F$ 
    
```

Let (G, c) be a weighted graph. We say that a subset $F \subseteq E(G)$ is *good* if there exists a minimum spanning tree T of (G, c) such that $F \subseteq E(T)$.

Theorem 3.5. *Let (G, c) be a weighted graph. If F is good and K is a nonempty cut of G with $K \cap F = \emptyset$, then $F \cup \{e\}$ is also good, where $e \in K$ is such that*

$$c(e) = \min\{c(f) \mid f \in K\}.$$

Proof. Suppose that F is good and that K is a nonempty cut of G with $K \cap F = \emptyset$. Let $e \in K$ be such that

$$c(e) = \min\{c(f) \mid f \in K\}.$$

Since F is good, there exists a minimum spanning tree T of (G, c) such that $F \subseteq E(T)$. If $e \in E(T)$, then $F \cup \{e\} \subseteq E(T)$, and therefore $F \cup \{e\}$ is good.

Assume then that $e \notin E(T)$. Let $X \subseteq V(G)$ be such that $K = \delta(X)$. Let x be the endpoint of e in X , and let y be the endpoint of e in $V(G) \setminus X$. Since T is connected, there exists an (x, y) -path

$$\langle u_0, \dots, u_k \rangle$$

in T . Now $u_0 = x \in X$ and $u_k = y \in V(G) \setminus X$. Hence there exists $i \in [0 : k)$ such that $u_i \in X$ and $u_{i+1} \in V(G) \setminus X$. Set Then $f := u_i u_{i+1} \in K$. Since e has minimum cost among the edges of K , we have $c(e) \leq c(f)$. Consider $T' := T - f + e$. By Lemma 3.4, the graph T' is a tree. Since $V(T') = V(T) = V(G)$ and $E(T') \subseteq E(G)$, it follows that T' is a spanning tree of G . Moreover,

$$\tilde{c}(T') = \tilde{c}(T) - c(f) + c(e) \leq \tilde{c}(T).$$

Since T is a minimum spanning tree of (G, c) , it follows that T' is also a minimum spanning tree of (G, c) . Finally, $f \notin F$, because $f \in K$ and $K \cap F = \emptyset$. Hence $F \subseteq E(T')$. Also $e \in E(T')$. Therefore $F \cup \{e\} \subseteq E(T')$. Thus $F \cup \{e\}$ is good. $\square$

Theorem 3.6. *Let (G, c) be a weighted graph. If $F \subseteq E(G)$ is good, then, for each $e \in F$, there exists a cut K of G such that $e \in K$, $K \cap (F \setminus \{e\}) = \emptyset$, and*

$$c(e) = \min\{c(f) \mid f \in K\}.$$

Proof. Assume that F is good, and let $e \in F$. Since F is good, there exists a minimum spanning tree T of (G, c) such that $F \subseteq E(T)$. Let x and y be the endpoints of e , and set

$$X := [x]_{T-e}.$$

We claim that $y \notin X$. Indeed, if $y \in X$, then $T - e$ contains an (x, y) -path. Adding the edge $e = xy$ to this path, we obtain a circuit in T , a contradiction. Thus $y \in V(T - e) \setminus X$.

Now, by the definition of X , we have $\delta_{T-e}(X) = \emptyset$. Since e has one endpoint in X and the other in $V(T - e) \setminus X$, it follows that $\delta_T(X) = \{e\}$. Let $K := \delta_G(X)$. Then K is a cut of G , and $e \in K$. Moreover, since $F \subseteq E(T)$ and $\delta_T(X) = \{e\}$, we have $K \cap (F \setminus \{e\}) = \emptyset$.

It remains to prove that $c(e) \leq c(f)$ for every $f \in K$. Let $f \in K$. If $f = e$, there is nothing to prove. Suppose then that $f \neq e$. Since $f \in K = \delta_G(X)$, the edge f has one endpoint in X and the other in $V(G) \setminus X$. Also, since $\delta_T(X) = \{e\}$, the edge e belongs to the unique path in T joining the endpoints of f . Therefore, by Lemma 3.4, the graph $T' := T - e + f$ is a tree. Since $V(T') = V(T) = V(G)$ and $E(T') \subseteq E(G)$, it follows that T' is a spanning tree of G . Because T is a minimum spanning tree of (G, c) , we have

$$\tilde{c}(T) \leq \tilde{c}(T') = \tilde{c}(T) - c(e) + c(f).$$

Hence $c(e) \leq c(f)$. Therefore $c(e) = \min\{c(f) \mid f \in K\}$. $\square$

Corollary 3.7. *Let (G, c) be a weighted connected graph. Algorithm 3.2 returns the edge set of a minimum spanning tree of (G, c) .*

Proof. By the correctness of Algorithm 3.1, Algorithm 3.2 executes $|V(G)| - 1$ iterations and returns a set $F \subseteq E(G)$ such that $(V(G), F)$ is a spanning tree of G .

It remains to show that this spanning tree has minimum weight. We prove, by induction on the number of iterations, that F is good at the beginning of every iteration. At the beginning, $F = \emptyset$, which is good because G has at least one minimum spanning tree. Suppose now that F is good at the beginning of an iteration. The algorithm chooses a nonempty cut K with $K \cap F = \emptyset$, and then chooses an edge $e \in K$ of minimum weight in K . By the previous theorem, $F \cup \{e\}$ is good. Thus the invariant is preserved.

At the end of the algorithm, $(V(G), F)$ is a spanning tree of G . Since F is good, there exists a minimum spanning tree T of (G, c) such that $F \subseteq E(T)$. But both $(V(G), F)$ and T are spanning trees of G , and therefore both have $|V(G)| - 1$ edges. Hence $F = E(T)$. Thus $(V(G), F)$ is a minimum spanning tree of (G, c) . $\square$

3.3 Kruskal-Boruvka algorithm

Algorithm 3.3 KRUSKAL(G, c)

```

1:  $F \leftarrow \emptyset$ 
2: order the edges of  $G$  as  $e_0, \dots, e_{m-1}$  so that  $c(e_0) \leq c(e_1) \leq \dots \leq c(e_{m-1})$ 
3: for  $i \leftarrow 0$  to  $m - 1$  do
4:   if  $(V(G), F \cup \{e_i\})$  is acyclic then
5:      $F \leftarrow F \cup \{e_i\}$ 
6: return  $F$ 

```

Theorem 3.8. *Let (G, c) be a weighted connected graph. Algorithm 3.3 returns a set $F \subseteq E(G)$ such that $(V(G), F)$ is a minimum spanning tree of (G, c) .*

Proof. Let $e_0, \dots, e_{m-1}$ be the order in which Algorithm 3.3 considers the edges of G . Thus

$$c(e_0) \leq c(e_1) \leq \dots \leq c(e_{m-1}).$$

For each $j \in [0 : m]$, let $E_j := \{e_k \mid k \in [0 : j]\}$. We first prove that, throughout the execution of the algorithm, it holds that

- (i) the set F is good, and
- (ii) for each $xy \in E_i \setminus F$, there is a (x, y) -path in the graph $(V(G), F)$.

At the beginning, $F = \emptyset$, which is good because G is connected and therefore has a minimum spanning tree; moreover, $E_0 \setminus F = \emptyset$.

Suppose now at the beginning of an iteration of the for-loop, we have that F is good and that for each $xy \in E_i \setminus F$, there is a (x, y) -path in the graph $(V(G), F)$. We have two cases to consider.

First suppose that $(V(G), F \cup \{e\})$ is not acyclic. We must prove that the invariant is preserved, that is, that

(1.i) F is good, and

(1.ii) for each $xy \in E_{i+1} \setminus F$, there is a (x, y) -path in the graph $(V(G), F)$.

In this case (1.i) is trivial, since it holds by hypothesis. For (ii), let x and y be the endpoints of e_i . Because $(V(G), F \cup \{e\})$ is not acyclic, we have that the graph $(V(G), F)$ contains an (x, y) -path. This proves (1.ii).

Assume then that $(V(G), F \cup \{e_i\})$ is acyclic. We must now prove that

(2.i) $F \cup \{e_i\}$ is good, and

(2.ii) for each $xy \in E_{i+1} \setminus (F \cup \{e_i\})$, there is a (x, y) -path in the graph $(V(G), F)$.

Let $H := (V(G), F)$, and let $X := [x]_H$. Then $\delta_H(X) = \emptyset$. Let $K := \delta_G(X)$. Notice that $e_i \in K$. We show now that $c(e_i) = \min\{c(f) \mid f \in K\}$. Let $f \in K \setminus \{e_i\}$. We claim that $f = e_j$ for some $j \in (i : m)$. To see this, observe that if $uv \in E_i \setminus F$, then H contains a (u, v) -path and so any cut that contains uv also contains an edge of F . Hence $K \cap (E_i \setminus F) = \emptyset$. Since $K \cap F = \emptyset$, it follows that $f = e_j$ for some $j \in (i : m)$ and thus $c(e_i) \leq c(f)$. By Theorem 3.5, it follows that $F \cup \{e_i\}$ is also good, and so (2.i) holds. For (ii), notice that $E_{i+1} \setminus (F \cup \{e_i\}) = E_i \setminus F$, and thus (2.ii) holds by hypothesis.

It remains to prove that the final set F defines a spanning tree. Since F is good, we have that $H := (V(G), F)$ is acyclic. Moreover, for every edge $xy \in E(G) \setminus F$, there exists a (x, y) -path in H . We claim that H is connected. Let $u \in V(H) = V(G)$ and consider the set $X := [u]_H$. Let $x \in X$ and suppose that $xy \in E(G) \setminus F$. Since H contains a (x, y) -path, we get that $y \in X$. Thus $\delta_G(X) = \emptyset$. Because G is connected, this implies that $X = \emptyset$ or $X = V(G)$. Since $X \neq \emptyset$, we have that $X = V(G)$. Therefore H is connected. $\square$

3.4 Exercises

Exercise 3.1

Let T_1 and T_2 be spanning trees of a graph G . Show that, for each $e \in E(T_1) \setminus E(T_2)$, there exists $f \in E(T_2) \setminus E(T_1)$ such that both

$$T_1 - e + f \quad \text{and} \quad T_2 - f + e$$

are spanning trees of G .

Exercise 3.2

Let (G, c) be a weighted graph. Show that, if T and F are minimum spanning trees of (G, c) , then the lists of the weights of the edges of T and F , ordered in nondecreasing order of weight, are identical.

Exercise 3.3

Let (G, c) be a weighted connected graph, and let $\emptyset \neq X \subseteq V(G)$. Prove or disprove the following statements.

- (i) If T is a minimum spanning tree of (G, c) and $T[X]$ is connected, then $T[X]$ is a minimum spanning tree of $(G[X], c)$.
- (ii) If J is a minimum spanning tree of $(G[X], c)$, then $E(J)$ is good for (G, c) .

Exercise 3.4

Let (G, c) be a weighted connected graph such that $c(e) \neq c(f)$ for all distinct $e, f \in E(G)$. Show that (G, c) has a unique minimum spanning tree.

Exercise 3.5

Let (G, c) be a weighted connected graph. Give an algorithm that computes the set of all edges that belong to some minimum spanning tree of (G, c) . Prove the correctness of your algorithm and determine its time complexity.

Exercise 3.6

Let (G, c) be a weighted connected graph. Show that (G, c) has a unique minimum spanning tree if every nonempty cut K of G contains a unique edge $e \in K$ such that

$$c(e) = \min\{c(f) \mid f \in K\}.$$

Show that the converse does not hold.

Exercise 3.7

Let (G, c) be a weighted connected graph, and let T be a spanning tree of G . Define

$$b(T) := \max\{c(e) \mid e \in E(T)\}.$$

An edge $e \in E(T)$ such that $c(e) = b(T)$ is called a *bottleneck edge* of T . A spanning tree T of G is called a *minimum-bottleneck spanning tree* of (G, c) if $b(T)$ is minimum among all spanning trees of G .

Prove or disprove each of the following assertions.

- (i) If T is a minimum-bottleneck spanning tree of (G, c) , then T is a minimum spanning tree of (G, c) .
- (ii) If T is a minimum spanning tree of (G, c) , then T is a minimum-bottleneck spanning tree of (G, c) .

Index

- (v_0, v_k) -walk, 37
- X -good, 27
- d -separated, 19
- (v_0, v_k) -walk, 6
- acyclic, 8
- adjacent, 2
- alphabet, 44
- binary encoding, 44
- binary tree, 54
- capacity, 58
- characteristic function of X relative to U , 3
- circuit, 7
- closed, 6
- code, 44
- codeword, 44
- compatibility digraph, 38
- compatible, 23
- compatible partition, 39
- concatenation, 6, 44
- conflict graph, 32
- connected, 8
- connected components, 8
- contraction, 47
- cost, 44
- cover, 63
- covering, 16
- covers, 16
- cut, 2, 12
- cycle, 37
- degree, 2
- deletion, 4
- digraph, 36
- edge-induced, 4
- edges, 1
- efficiency, 58
- empty, 6, 44
- endpoints, 1
- equal-length interval covering problem, 16
- expansion, 47
- feasible, 59
- feasible matching, 63
- fractional knapsack, 58
- fractional knapsack instance, 58
- fractional knapsack problem, 59
- frequency, 44
- frequency table, 44
- full, 54
- good, 69
- graph, 1
- incompatible, 23
- indegree, 54
- induced, 4
- internal vertex, 54
- items, 58

- label, 54
- leaf, 54
- left arc, 54
- length, 6, 37

- maximum compatible subset of
 (T, s, f) , 23
- maximum compatible subset of T ,
 23
- minimum covering, 16
- minimum prefix-free binary
 encoding, 44
- minimum spanning tree, 68
- minimum spanning tree problem,
 68

- neighbor, 2
- neighborhood, 2
- neighbors, 2

- optimal in (T, s, f) , 23
- optimal in T , 23
- ordered binary tree, 54
- outdegree, 54

- partial row cover, 63
- partition, 31
- path, 7, 37
- prefix, 44
- prefix ordered digraph, 55
- prefix-free, 44
- proper prefix, 44

- reachability, 7
- right arc, 54
- root, 54
- row cover, 63

- schedule, 64
- sequence, 6
- set of arcs, 36
- set of vertices, 36
- siblings, 44
- spanning, 4
- spanning tree, 65
- subgraph, 4
- substructure induced by X , 23
- supergraph, 4
- symbols, 44

- task scheduling problem, 23
- task structure, 23
- tasks, 23
- transitive reduction, 39
- tree, 10

- uses, 6

- value, 58, 59, 64
- vertex-induced, 4
- vertices, 1

- walk, 6, 37
- weight, 58, 59, 68
- weighted graph, 68
- word, 44, 55

Index of notation

$\mathbb{1}, 3$	$G + e, 5$
$\deg_G, 2$	$G + F, 5$
$\delta_G, 2$	$G - F, 5$
$E(G), 1$	$G[X], 4$
$G[F], 4$	$N_G, 2$
$G - u, 4$	$T(x, y), 10$
$G - X, 4$	$V(G), 1$